



Universidad
Carlos III
de Madrid

Master's Degree in Robotics and Automation

Academic Year 2020/2021

Master's Thesis

“Application based on convolutional neural networks for prediction of brain tumours through magnetic resonance images”

Student: Diego L. Carrillo Santamaría

Supervisor: Arturo de la Escalera Hueso

Madrid, October 2021

DETECCIÓN DEL PLAGIO

La Universidad utiliza el programa **Turnitin Feedback Studio** para comparar la originalidad del trabajo entregado por cada estudiante con millones de recursos electrónicos y detecta aquellas partes del texto copiadas y pegadas. Copiar o plagiar en un TFM es considerado una **Falta Grave**, y puede conllevar la expulsión definitiva de la Universidad.



[Incluir en el caso del interés de su publicación en el archivo abierto]

Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento – No Comercial – Sin Obra Derivada**

ABSTRACT

This study investigates the use of artificial intelligence (AI) to support the detection, classification and prediction of brain tumours. It focuses on artificial neural networks for the classification and analysis of medical images acquired using established imaging techniques.

The project develops an application designed to support clinicians specialising in neuroscience, radiology and oncology. The system uses datasets obtained from healthcare institutions through magnetic resonance imaging (MRI) and is intended to assist with the diagnosis of patients who may have a brain tumour.

On the other hand, the results of the implementation of said neural networks will be presented using “open source” software such as Python with the Keras and TensorFlow libraries in development environments such as Anaconda and Google Colab which are freely available for such purposes. Likewise, the different datasets available to the community of developers and researchers are shown, as well as their use and treatment in the qualitative analysis of the results.

Several implementation approaches are evaluated in terms of performance and predictive accuracy to identify a reliable solution. A web application is also developed to facilitate tumour-type prediction, a challenging task because of the physical characteristics and variability of these tumours. The application presents the results through an accessible visual interface.

The workflow incorporates image-processing and data-augmentation techniques, including colour adjustment, heat maps, resizing and rotation. These techniques support effective model training and help clarify the behaviour of the system, whose primary objective is to identify the type of brain tumour that may be present in a patient.

Modern healthcare technologies require high levels of reliability, accuracy and usability. Accordingly, this thesis applies the knowledge acquired during the Master's programme to a socially relevant problem and seeks to contribute to the development of tools that can support patients and healthcare professionals in addressing a complex and potentially devastating disease.

KEYWORDS

Artificial intelligence, artificial neural networks, machine learning, deep learning, dataset, brain tumour, glioma, meningioma, pituitary tumour, magnetic resonance.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my family and closest friends for their encouragement and unwavering support. Their understanding and confidence in my decision to undertake this demanding and rewarding journey have had a profound influence on both my professional development and my personal growth.

I am equally grateful to the academic staff, lecturers and fellow students who accompanied me throughout the Master's programme. Their academic guidance, professional insight and personal generosity enriched this experience immeasurably. Without their contribution, this journey would have been considerably more difficult.

TO ALL, THANK YOU VERY MUCH!

TABLE OF CONTENTS

CHAPTER I: INTRODUCTION	8
CHAPTER II: OBJECTIVES	9
CHAPTER III: JUSTIFICATION OF THE PROBLEM	10
CHAPTER IV: STATE OF THE ART	11
4.1 INTERDISCIPLINARY MODALITY OF MEDICINE AND ARTIFICIAL INTELLIGENCE	11
4.2 MEDICAL STUDY OF THE MAIN BRAIN TUMOURS	12
4.3 USE OF CNN FOR MEDICAL IMAGE RECOGNITION.	12
CHAPTER V: TOOLS USED IN IMPLEMENTATION	13
5.1 GOOGLE COLAB	13
5.2 GOOGLE DRIVE	13
5.3 JUPYTER NOTEBOOK	13
5.4 TENSORFLOW	14
5.5 KERAS	14
5.6 NEURAL NETWORK	15
5.7 CONVOLUTIONAL NEURAL NETWORK (CNN)	16
5.8 DATASETS	16
5.9 BIG DATA	16
5.10 FLASK	17
CHAPTER VI: QUALITATIVE STUDY OF THE MAIN TYPES OF TUMOR	18
6.1 MENINGIOMAS	18
6.2 PITUITARY TUMOUR	19
6.3 GLIOMA	20
6.4 IMAGES OBTAINED BY MAGNETIC RESONANCE (MRI).	20
6.5 NATURE OF A BRAIN TUMOUR	21
CHAPTER VII: SYSTEM DEVELOPMENT AND IMPLEMENTATION	24
7.1 DATASETS USED IN IMPLEMENTATION	24
7.1.1 IMPROVED PERFORMANCE DATASET	24
7.1.2 DATASET KAGGLE	24
7.1.3 GITHUB DATASETS	24
7.2 DATASET PREPARATION	24
7.3 OVERFITTING	25
7.4 USE OF PRE-TRAINED CNNs AND TRANSFER LEARNING	25
7.5 THE CONVOLUTION	26
7.6 VGG16	27
7.7 INTEGRATION OF THE GOOGLE COLAB ENVIRONMENT	28
CHAPTER VIII: EXPERIMENTAL RESULTS	30
8.1 USE OF GRAPHIC PROCESSING UNIT (GPU)	30
8.2 INITIAL PARAMETERS	32
8.3 USE OF LIBRARIES	32
8.4 INITIAL TESTS	33
8.5 IMPROVED TRAINING PROCESS	38
8.6 DATA AUGMENTATION	44
8.7 USE OF TRANSFER LEARNING	45
8.8 COMPARATIVE ANALYSIS WITH OTHER STUDIES	50
CHAPTER IX: APPLICATION DEVELOPMENT	53
9.1 PRE-PROFESSIONAL FEEDBACK	53
9.2 APPLICATION DEVELOPMENT	53
9.2.1 BACK-END APPLICATION DEVELOPMENT.	54
9.2.2 FRONT-END APPLICATION DEVELOPMENT	55
9.2.3 EXECUTION OF THE WEB APPLICATION.	56
9.2.4 VISUAL APPLICATION EXECUTION	59
CONCLUSIONS.	63
NEXT STEPS TO TAKE	64
REFERENCES	65

LIST OF FIGURES

Figure 1: Main areas of medical sciences where AI is applied.	11
Figure 2: Google Colab integration with other services and user interfaces.	13
Figure 3: TensorFlow toolkit hierarchy.	14
Figure 4: Keras hierarchy order	14
Figure 5: Basic model of a neural network.	15
Figure 6: Fundamental architecture of a NN.	15
Figure 7: General representation of the architecture of a CNN	16
Figure 8: The three "V's" of Big Data according to Gartner.	16
Figure 9: Example of integration using the Flask framework.	17
Figure 10: Incidence graph by age and sex per 100,000 inhabitants in the United States between 2002 and 2006.	18
Figure 11: Magnetic resonance imaging (MRI) of a brain meningioma.	18
Figure 12: Superior and lateral view of pituitary microadenoma located at the base of the pituitary gland.	19
Figure 13: Images obtained by MRI of a pituitary tumor A) Top view B) Left lateral view.	19
Figure 14: Computed Tomography Image of a Glioma located in the axial lobe.	20
Figure 15: Physical interpretation of the magnetic field created by the generating device during the image acquisition process.	21
Figure 16: Graphic representation of an MRI machine	21
Figure 17: Anatomical similarity of a Meningioma and Pituitary tumor.	21
Figure 18: a) Early and Advanced phase of a Glioma brain tumour. b) Real image of the removal process of an advanced Glioma tumor in a patient	23
Figure 19: folder distribution in Google Drive.	24
Figure 20: Internal structure of each of the folders for Training, Validation and Testing.	25
Figure 21: Behavior of an overfitting function.	25
Figure 22: Convolution applying a 3 x 3 pixel filter for a 5 x 5 pixel image.	26
Figure 23: Performance of a convolution on an image	27
Figure 24: Architecture of the VGG16 model with 2 fully connected layers.	27
Figure 25: Summary table of the VGG16 architecture.	28
Figure 26: Google Drive integration with the Google Colab environment.	28
Figure 27: Integration message successfully completed.	29
Figure 28: Example of storing datasets for training, Meningioma tumor type.	29
Figure 29: GPU information provided by Google Colab.	30
Figure 30: NVIDIA T4 GPU technical specifications.	31
Figure 31: Sampling of GPUs available to start code execution.	31
Figure 32: Tesla K80 GPU allocation in new Google Colab session. Source: Google Colab (Own creation)	31
Figure 33: Effect on learning performance using the ADAM optimizer for different batch sizes.	32
Figure 34: Use of TensorFlow and Keras libraries for system development.	32
Figure 35: Total images chosen for Training, Validation and Test, for both classes.	33
Figure 36: Application of RGB filter to extract image features with class prediction.	33
Figure 37: Image classification according to class.	33
Figure 38: Summary of the model created in the initial test.	34
Figure 39: Block diagram of created model.	34
Figure 40: Sigmoid activation function*	35
Figure 41: ReLU activation function**	35
Figure 42: Tanh activation function.	36
Figure 43: Accuracy in validation and behavior of losses during the training process.	36
Figure 44: Precision graph in the training and validation process.	36
Figure 45: Loss graph during the training and validation process.	37
Figure 46: Arrangement of a confusion matrix	37
Figure 47: Confusion matrix for the Meningioma and NO_Tumor classes.	38
Figure 48: Matplotlib function to obtain block neural network model.	38
Figure 49: Neural network architecture by layers and blocks.	38
Figure 50: MRI of Gliomas with different resolutions.	40
Figure 51: Pre-processed images before starting the training of the neural network. A) Brain with tumor in lateral and frontal view. B) Brain without presence of tumors in lateral and frontal view.	41
Figure 52: Experimentation with new dataset.	41
Figure 53: Summary of the model created in the second test with a larger dataset.	41
Figure 54: Neural network class labels.	42
Figure 55: Distribution of images by class.	42
Figure 56: Confusion matrix for the Meningioma and NO_Tumor classes with new dataset.	43
Figure 57: Precision graph of the training and validation processes.	44
Figure 58: Loss graph of the training and validation processes.	44
Figure 59: Configuration of the data augmentation function of our cnn.	45
Figure 60: Result of applying data augmentation on an image.	45
Figure 61: Transfer Learning model applied to new model.	46
Figure 62: Dataset of 4 classes for implementation with Transfer Learning.	46
Figure 63: Labeling by classes for Transfer Learning	46
Figure 64: VGG model used for Transfer Learning.	47
Figure 65: Last three layers of the VGG16 model	47
Figure 66: Last 3 layers of the VGG16 model with output layer adjustment.	47
Figure 67: Training performance in accuracy and losses with transfer learning.	47

Figure 68: a) Confusion matrix for 4-class CNN., b) Evaluation of accuracy in the training process.	48
Figure 69: Performance graphs in accuracy and losses for the cnn training and validation process.	49
Figure 70: Similarity between images of different classes.	49
Figure 71: Views in MRI images from different angles in datasets.	49
Figure 72: a) Confusion Matrix obtained in previous study. B) Percentage of true positive readings in our system predictions.	51
Figure 73: Neural Network Model applied in study.	51
Figure 74: Operating diagram of the proposed application.	54
Figure 75: Flask application and neural network model repository.	54
Figure 76: Image pre-processing within the Flask application called app_cnn_predict4.py	54
Figure 77: Enabling Flask environment.	55
Figure 78: Access to the jQuery library to call the "prediction" function	55
Figure 79: Publication in a web environment in the app written in HTML.	55
Figure 80: HTML application repository within our master repository.	55
Figure 81: Access to the web application.	56
Figure 82: Predictions obtained in our web application.	56
Figure 83: Enabling library for D3js functionality for graphical application.	59
Figure 84: Visual application for class prediction. Example with an MRI image without tumor.	59
Figure 85: Numerical results of predictions on the same image used in the previous example.	59
Figure 86: Visual application for a Pituitary Tumour with similarity to other classes.	61
Figure 87: Numerical analysis of predictions for the same MRI image with Pituitary Tumour.	62

CHAPTER I: INTRODUCTION

During recent years, the rise of Artificial Intelligence applied in different areas of science and industry has evolved to extraordinary levels and specifically the field of medical sciences has not been the exception. As is known, in particular the topic of image processing has come to significantly help in the research and development of technologies and techniques that collaborate to make processes in the medical area not only much more efficient, but also effective and reliable.

This generates a positive impact on the quality of life of people, mainly those who suffer from diseases of the central nervous system and especially we are talking about brain tumours, whose impact on the patient's health can be lethal if not detected or treated in time. According to the Association of People Affected by Brain Tumours in Spain (ASATE), in the world the incidence of brain tumours is 7.5 per 100,000 inhabitants and they are considered the second cause of death in children under 5 years of age. Although it is recognized that 90% of tumors are benign, the remaining 10% of cases can become lethal tumors due to their nature since they directly attack the tissue of one of the most delicate and sensitive organs of the being. human, as is the brain. (ASATE, 2021).

One of the most notable characteristics of this disease is that the cause that generates it is openly unknown, therefore, its prediction and diagnosis can often be complex. For this reason, we have considered the idea of exploring this field, its complexity makes it abstract and often undefined, not only in how but in why it is presented and in what way it is done, knowing that the techniques to collect information are mainly the use of medical images obtained by already well-known methods such as: magnetic resonance imaging (MRI), computed axial tomography (CAT), Single-Photon Emission Computed Tomography (SPECT) and Positron Emission Tomography (PET), the results of which can be used for analysis. computing using AI techniques (García Fenoll, 2010).

Furthermore, this study aims to focus on the analysis of the main types of brain tumours that are diagnosed by medical personnel in the field of neurology, radiology and oncology, and who detect a considerable number of new cases in Spain each year, a situation that has served as a basis for researching and developing tools that support the diagnosis, and that in turn, help in the patient recovery process, thanks to the use of AI techniques for computer image processing that allow generating predictive models.

Thanks to these research works that have been recently developed in this area, it has been possible to stabilize and even reduce mortality in most countries. Early detection and correct treatment of the disease are the fundamental pillars to combat said mortality in people of all ages in which this condition can manifest. Clearly, there is still a lot of work to do since a large number of cases still occur. Taking advantage of all the technological advantages that we have available today allows us to delve deeper into the study of diseases, and in this case, in the applications that support the field of neurological and radiological sciences.

CHAPTER II: OBJECTIVES

The development of this project has the intention of creating an application for the detection and classification of brain tumours using medical Magnetic Resonance images, using Artificial Neural Networks for image classification.

- Develop an application for the multiclass detection of brain tumours through the use of medical images and predefined datasets.
- Build a prediction platform for brain tumours: Meningioma, Glioma, Pituitary and Non-Tumor, whose incident cases are the most frequent in specialized health centers.
- Implement the use of image classification techniques through Neural Networks in order to predict the type of brain tumour presented in the image.
- Develop a web application that is capable of reproducing the Neural Network model created, which allows showing the predictions made by the implemented CNN model.
- Use the Google Colab platform for the development of the algorithm, as well as all the functions it offers, mainly the use of the GPU provided for free.
- Develop functions facilitated by TensorFlow and Keras for the implementation of the proposed neural network.

CHAPTER III: JUSTIFICATION OF THE PROBLEM

As mentioned above, brain tumours are widely heterogeneous diseases and can be considered from simple benign tumours that may or may not require surgical intervention to fatal tumours depending on their nature, size, shape, growth and region where they develop. The simple fact that the origin of this type of tumour is unknown for certain has made it the subject of research and study for some years now.

In current times, with the technological deployment and development to which we have access, as well as the tools offered for the analysis and multidisciplinary study of this disease, health authorities around the world and of course, in Spain, have provided the necessary resources for the study and analysis of this phenomenon, since it represents a significant percentage of people who are affected by this disease and for which special attention must be paid, since health is a fundamental right of the human being that belongs to the individual, society and the state.

One of the main challenges of exploring this field is to create a sufficiently reliable prediction system that shows safe results, because medical specialists must be extremely cautious when making a diagnosis, since the subsequent treatment of the patient depends on it. A failure to diagnose can have very negative consequences for those who suffer from this disease.

That is why different techniques must be explored that allow us to generate a safe and reliable prediction model, today we have a wide range of CNN models that have been created from previous studies, the idea of this research work is not only limited to the creation of the system, but also aims to build an application that allows in a simple and simple way to be able to evaluate real images and from there generate predictive models that help medical personnel in the task of detection and diagnosis, taking into consideration that we are facing a complex and diverse scenario, since there are no defined patterns in the way tumours present.

CHAPTER IV: STATE OF THE ART

4.1 INTERDISCIPLINARY MODALITY OF MEDICINE AND ARTIFICIAL INTELLIGENCE

Research and development in the field of medical sciences using AI to *create* applications has been showing a significant increase in recent years, thanks to the infinite resources provided for the diagnosis, treatment and recovery of diseases. In this work, the focus is on the area of neuroscience, specifically neurology in combination with other disciplines such as radiology and oncology.

As is known, the dense world of image processing using AI techniques is a fundamental pillar for the analysis of medical images obtained by different methods such as magnetic resonance imaging (MRI), computed axial tomography (CAT), Single Photon Emission Computer Tomography (SPECT) and Photo *Emission* Tech (PET), since *thanks* to these procedures it is possible to determine and diagnose the state in which a patient is, and from this assessment it is possible to evaluate the treatment to which they should be subjected.

Strictly speaking, image analysis is based on logical and mathematical models and allows us to understand different neurocognitive mechanisms that underlie not only the understanding of biological activity, but also the use of technologies such as magnetoencephalography (MEG) that facilitate the analysis of brain activity practically in real time (Braininvestigators, 2018).

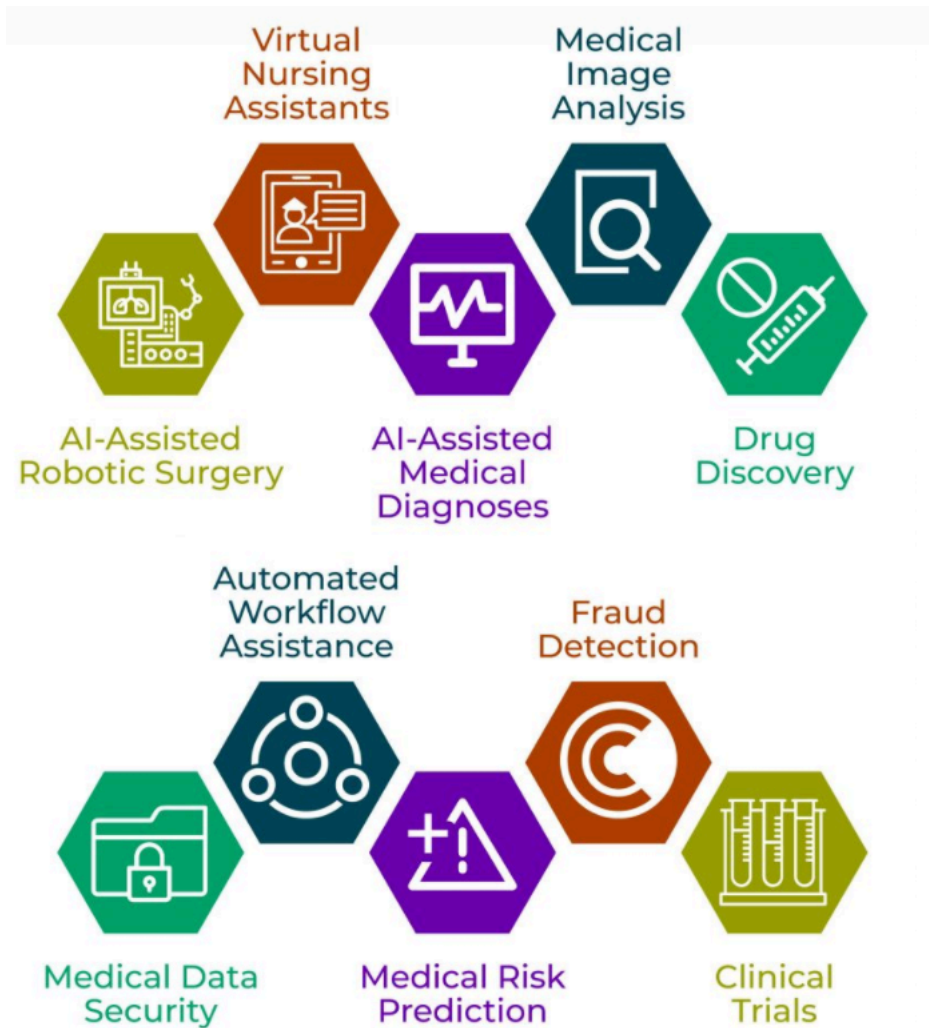


Figure 1: Main areas of medical sciences where AI is applied.

Source: Ignite R&D Outsourcing (Available at: <https://igniteoutsourcing.com/healthcare/artificial-intelligence-in-healthcare/>)

As shown in Figure 1, we *observe* that there are 10 main areas where AI is applied in the medical field. For this research development, we will focus on the area of Medical Image Analysis and Assisted Medical Diagnostics, and in which a multidisciplinary vision must be used because technological concepts serve as a basis to support science and the development of applications that make their work easier for medical personnel.

This means an enormous contribution since it will greatly facilitate a task that is not only complex, but also usually requires a lot of time, since a rigorous analysis is required that is always prone to human error, and in which large amounts of economic and professional resources are invested to be able to give an effective diagnosis.

4.2 MEDICAL STUDY OF THE MAIN BRAIN TUMOURS

Brain tumors represent a significant percentage of causes of death in many countries around the world, which shows a high social impact due to their poor prognosis, which has caused health authorities in different countries to show special interest in the study and research of this disease that many people suffer from. Since the late 1980s and up to the present date, multiple studies have been developed that have been accompanied by the technological revolution of recent decades. In 2016, the WHO incorporated a new edition of the "Classification of primary tumors of the central nervous system" (World Health Organization, 2016). This consultation document has not only demonstrated the importance that public knowledge of these tumors has for the WHO, but also that it serves as support for the research of the community of doctors and health technologists.

In Spain, studies reveal that brain tumours cover approximately 2% of the total number of people with cancer in adults and 15% in children under 15 years of age. This rate is relatively low compared to other types of cancer and its mortality has been reduced with technological advances and new medical procedures for its treatment (Spanish Society of Medical Oncology, 2019). area of study of interest by the global medical community.

Another of the singularities of this disease is that it is unknown exactly which factors can cause urination, although it is believed that it could be linked to various factors such as viruses, genetic factors, radiation and consumption of chemical substances, among others. On the other hand, research work has shown that men, the white race and children are those who have the highest incidence rate.

The most important developments in recent years have focused on the creation of applications capable of classifying and predicting the most common brain tumours, relying on image databases that can be obtained by different methods such as magnetic resonance imaging (MRI) and computerized axial tomography (CAT), which are freely available. These data databases usually contain an immense amount of data in the form of images since they work on Big Data, this is of great importance since these CNNs require a large amount of images that allow us to train the network reliably and with very accurate and truthful results.

4.3 USE OF CNN FOR MEDICAL IMAGE RECOGNITION.

The main challenge in the use of AI methods for the analysis and study of images is that a high degree of precision in the results is usually required, since the diagnosis of the patient's disease depends on it, which is why techniques have been created for image classification through CNN models, which have been designed to provide high accuracy in learning and thus convert them into very precise and reliable models. This task is essential since it is a highly sensitive activity and must provide confidence not only to the medical staff but also to the patient.

In the field of neuroscience, it must be taken into account that due to the complexity in detection and diagnosis, hard work has been done in a multidisciplinary manner to make the most of the technological advantages that are recently available. Computer vision is, by far, one of the most used applications that have come to help this area, since it is practically the most reliable and least invasive way to obtain "inputs" that allow us to analyze and make approximations regarding diseases that affect the brain and the central nervous system.

Recent studies suggest the use of *Deep Learning* with pre-trained networks to create a robust image classification system that is much more efficient in terms of the time required for training and the precision of the results, this makes it much easier for us to obtain more truthful outputs. The most recent studies propose the implementation of CNNs such as VGG16 and 19, Inception V3, ResNet50 and EfficenNet, these being the most popular, however, not the only ones available.

It is important to highlight that this allows us to use a technique known as transfer learning, which greatly facilitates the task of training CNNs. This can be interpreted as a starting point in the treatment of our work, in which a reference is taken in the solution of a previous work to provide a solution to another similar problem, thus evidencing its flexibility, which allows the extraction of features from the images that we use in training.

For computer image processing, it is ensured that transfer learning provides an important advantage and is the decrease in training time directly impacting a decrease in the generalization of the error, since the weights are reused in the layers of our CNN model.

This is even better in cases where the pre-trained model has a significantly larger number of labels and inputs (Goodfellow, Bengio, & Courville, 2016). The main uses given to pre-trained models are linked to: image classification, extraction of independent and integrated features and as weight initialization.

An important work that has been done regarding the classification of Brain Tumours in MRI images is "Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network" (Milica M. & Barjatarovic, 2020) carried out by Milica M. Badža and Marko Č. Barjaktarovic, in which the Figshare dataset was used (Cheng, Brain Tumour Dataset Figshare, 2017), this study addresses the issue of the heterogeneity of MRI images of brain tumours and how these affect the predictions of the neural network. In the same way, results are demonstrated in the predictions, these results obtained serve as a parameter to evaluate our proposal that we will see in later chapters in detail.

CHAPTER V: TOOLS USED IN IMPLEMENTATION

During the development of the work, some software tools necessary to treat the problem were used; most of these are available free of charge on the web and whose sole purpose is to support the research. Reference is made to each of them below.

5.1 GOOGLE COLAB

The evolution of AI, mainly in the last 10 years in the world of computing and computer development, as well as its applications in a huge number of areas of science, have notably influenced the giant American corporation with headquarters in California to enable a powerful interface for the development of software programs and applications that use models based on Machine *Learning and Deep* learning.

This interface is based on the *Jupyter Notebook* environment and was created with the main objective of training models based on neural networks, the system offers graphic processing units (GPU) for image processing and the service is available in the "cloud" therefore it offers many advantages, among which we can mention:

- **Integration:** Since it allows us to work with different languages, libraries and examples previously designed by other developers. Likewise, it is friendly to the GitHub platform, whose popularity in recent years has placed it as one of the main web applications thanks to its collaborative functions for the development of software and apps in general.

Likewise, being a Google service, it can be integrated with other platforms such as Google Drive, where datasets, images, repositories, executable files, among others, can be stored for free. The free service offers up to 15 GB free, and if more storage space is required, the payment option is provided according to the user's needs.

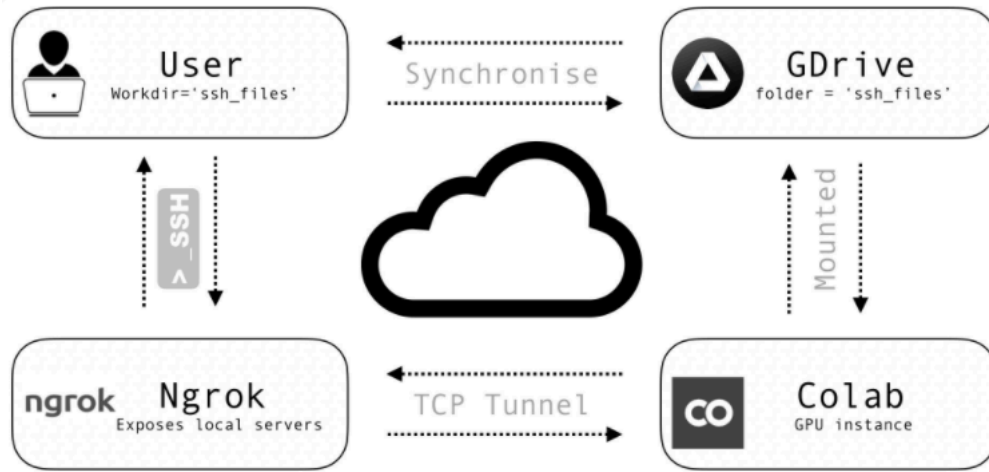


Figure 2: Google Colab integration with other services and user interfaces.

Source: Data Science blog of Imad El Hanafi (Available at: https://imadelhanafi.com/posts/google_colab_server/).

In *Figure 2* we see an example of an integration using Google Colab *services with Google Drive* and other third-party services. This is a great advantage in the implementation, since it avoids having to use the memory space of our personal computers and also avoids the loss of information since the service is available in the cloud at all times and files and repositories can be better protected. On the other hand, it is ideal for storing and using *datasets* to train neural networks.

- **Portability:** It offers us the ease of being able to work from any workstation, and even from different computers, because the service is offered in the cloud and only an internet connection is required to access it.
- **Efficiency:** Due to the fact that no special installation or configuration is required to use its services, the GPUs available are very powerful, which makes it an ideal tool for training neural networks and creating new models using the most popular languages.

5.2 GOOGLE DRIVE

It is another of the tools created *by Google* whose main purpose is to serve as cloud storage memory. In order to use this service, you only need to have a Google account, *which*, by the way, can be created for free. The space that this APP offers us is 15 GB, however, just as mentioned above, the capacity can be expanded by paying a monthly fee or by choosing one of the plans offered by Google. It is *important* to mention that it is not available in all countries, and it must also be taken into account that depending on the country, charges and taxes may be charged for its use. Currently you can have a maximum memory space of 2 TB.

The main advantage of this service is the ease of integration with all the services offered by *Google*, which makes it ideal for the implementation of the work in question, since by using the Google Colab *platform*, we can store the datasets *with* which we are going to work. On the other hand, it avoids the dependency of working with a single device, allowing us to work from any station just by having access to the internet.

5.3 JUPYTER NOTEBOOK

Another project created in order to support research and development among the community of programmers, developers and researchers by offering a completely free and open access environment, friendly to the Python environment and its applications. This *open source* is ideal for creating projects or sharing work and thus being able to make changes or improvements to other developments. Among the main advantages we can mention that it allows us

to execute code line by line, that is, we can monitor the response of each line of code that has been implemented, which allows us to detect errors or problems in execution instantly without having to carry out compilation and execution processes globally.

Jupyter Notebook, like Google *Colab*, requires internet access if special libraries or previously created *frameworks* are going to be used to integrate them into our code. This powerful tool is widely used by developers due to its excellent performance and reliability in the results, as well as the great ease it gives us in using apps that serve as support to give theoretical and graphical support to the projects.

5.4 TENSORFLOW

According to the official TensorFlow website, it is stated that “it is an end-to-end open source platform for machine learning” (TensorFlow, 2015). This platform allows us the flexible integration of a series of libraries for the development of projects and situations of an educational and research nature. Like many of the tools that are freely available to the developer community, it allows us to train models from the cloud and with a wide number of programming languages.

Its simple and flexible architecture has made it enormously popular today since it allows us to expand the concept of integration and complexity of the models we create, as well as its ease and speed of interacting with already existing models at a high level. This makes it easier for us to solve problems even when they are very complicated and with very high efficiency.

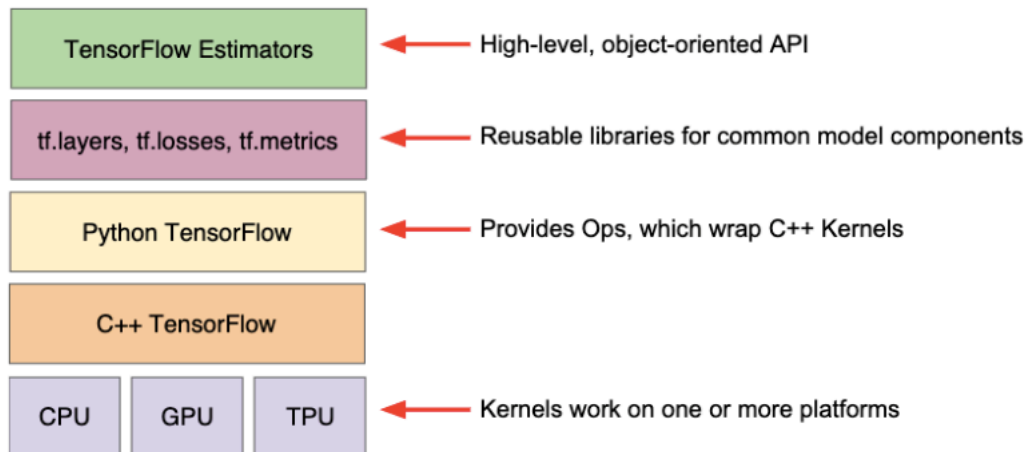


Figure 3: TensorFlow toolkit hierarchy.

Source: The Health Big Data Blog (Available at: <https://mksaad.wordpress.com/2020/01/14/tensorflow-toolkit-hierarchy/>).

Figure 3 shows us the toolkits with which TensorFlow has the capacity to work, from the highest object-oriented level to the use of hardware at the CPU, TPU and GPU level, that is, TensorFlow gives us the possibility of doing full integration with different computational levels.

5.5 KERAS

It is a popular Python library for neural network processing that works on the Machine Learning platform. Its use has been widely disseminated due to the great advantages and facilities it has for integrating with TensorFlow and Theano (Keras, 2015), which makes it a powerful tool with great capacity, speed and precision in the results. Its main advantage is that it is freely available and has an immense amount of documentation that makes it an ideal tool for research and development work.

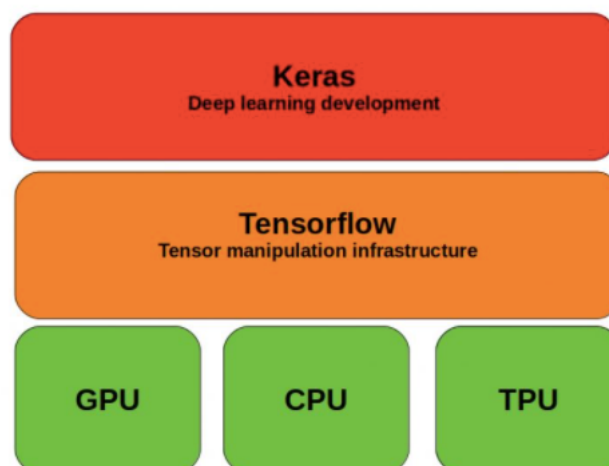


Figure 4: Keras hierarchy order

Source: The Health Big Data Blog (Available at: <https://starship-knowledge.com/awesome-python-data-science-libraries>).

Figure 4 shows us the hierarchy of Keras, its use is especially dedicated to working with Deep Learning models focused on a front end type experience to be developed with APIs and connect with web services. Likewise, Keras allows us to work with both special and conventional hardware with a wide number of datasets.

5.6 NEURAL NETWORK

Neural networks or NN (Neural Network) are computational patterns inspired by the structure of a human neuron, which represents an adaptive system that can be adjusted depending on the purpose for which it was designed (García Benítez, López Molina, Ramos Trejo., Palacios Morales, & Mora Méndez, 2016). NN are part of AI algorithms and their applications are used in countless technological fields that involve medical sciences, engineering, domestic and industrial applications, security and surveillance, among many others.

Its function is mainly oriented towards automatic learning of tasks based on a set of input data that can be tables with tabulated information, images, videos, audios, letters, numbers, etc., whose main function is to serve as a reference to provide information to the system so that it can learn from the input data automatically.

We can see the basic representation of a neural network in Figure 5, where we see a set of input data which varies depending on the application, as well as a series of interconnections that have implicit weights that determine the performance of the neuron. These weights can be interpreted as negative or positive depending on the inputs. On the other hand, there is an activation function that is usually non-linear capable of limiting the output amplitude.

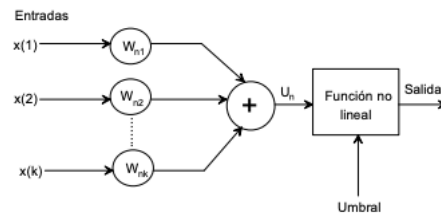


Figure 5: Basic model of a neural network.

Source: Artificial Neural Networks (p. 15).

The mathematical notation that describes the behavior of the output is obtained by the following equations:

Equation1	
-----------	--

and,

Equation 2	
$\text{Output} = \rho(U_n - \text{threshold})$	

where ρ is the activation function.

Architecture of a neural network:

Figure 6 shows the fundamental scheme of the architecture of a neural network:

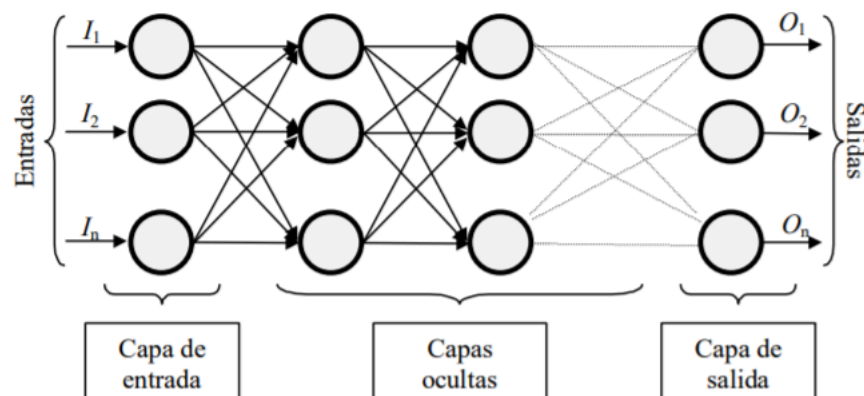


Figure 6: Fundamental architecture of a NN.

(Source: Neural Networks: Basic Concepts, page 12).

This structure shows a model segmented into layers, as follows:

- Input layer: where input data is received.
- Hidden layers: those that interconnect the input neurons with those of the output.

- Output layer: designed to extract the data already processed and display an output with the results obtained.

5.7 CONVOLUTIONAL NEURAL NETWORK (CNN)

Convolutional neural networks (CNN) are a type of neural network that has weights and biases that allow it to learn. This is done in such a way that a neuron receives one or more inputs, then applies a mathematical operation that performs a dot product and then an activation function.

Then we must take into account the losses which we will call the cost that acts in the last layer which is usually fully connected. CNNs are mainly used for image processing. The fundamental structure of CNNs is based on the convolutional layers, pooling or reduction and classifier layer (IAARbook, 2018).

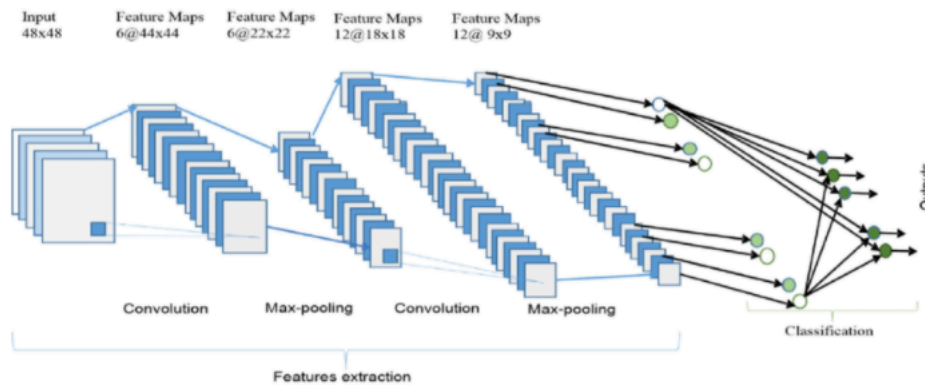


Figure 7: General representation of the architecture of a CNN

Source: Researchgate (Available at: https://www.researchgate.net/figure/The-overall-architecture-of-the-Convolutional-Neural-Network-CNN-includes-an-input_fig4_331540139)

Figure 7 shows us the basic architecture of a CNN, where there is an input with well-defined characteristics in terms of the weight and size of the image, and these are transferred to the subsequent hidden layers for extraction of characteristics of interest, applying special filters until reaching the last layer that is usually fully connected and this allows us to classify images from an input.

5.8 DATASETS

The datasets or dataset contain the necessary references to be able to train and process a neural network and normally these collections are usually presented in a tabulated configuration of data that can be numeral structures, texts, letters, sounds, images, videos or any reference that is going to be used to make the neural network automatically learn a pattern to obtain a result at the output.

Although there is no official definition, it can be assured that the data set is a representation of data housed in a memory which responds to a coherent and independent programming model, regardless of the origin of said data (Ozdemir & Susarla, Feature Engineering Made Easy, 2018).

5.9 BIG DATA

Big data refers to large-scale data sets with greater structural complexity derived from new sources, and in many cases hardware with special characteristics is required to be able to manage the computational weight for processing. As stated by the computer giant ORACLE, Big Data can be defined with the rule of the “three Vs”, referring to the Volume, Speed and Variety that Big Data manages in the processing and value it gives to neural network models to be able to train the neural networks. (Oracle Inc., 2016)

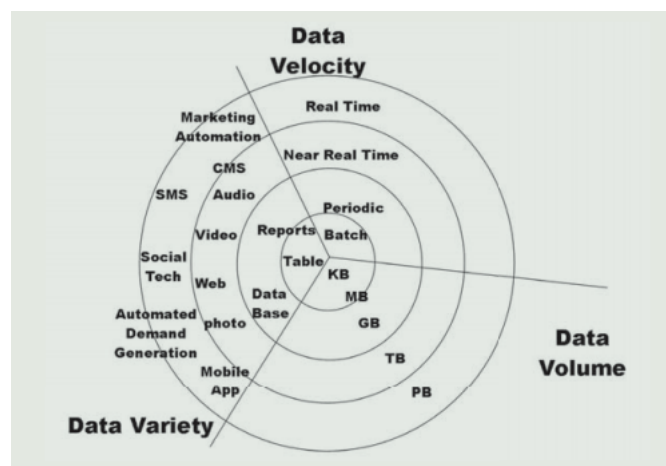


Figure 8: The three "V's" of Big Data according to Gartner.

Source: Big Bunker Data. (Available at: <https://thebigbunkerdata.wordpress.com/2015/04/01/las-3vs-del-big-data/>).

As we see in Figure 8, the *given* definition is shown; The variety is given by the extensive amount of data and information in different technological and scientific fields, the speed of their processing is also a fundamental variable, since modern technological demands demand speed in the processing of information and, finally; the volume or storage and processing capacity of the information with which you are working.

5.10 FLASK

Flask is a *framework* that has become popular in recent years due to its minimalist property whose root is written in *Python*, it does not require other tools or libraries in particular, but it does allow working with extensions and applications for object-oriented integrations.

Flask allows us to create web applications and integrations using its simple architecture, which is why it is known as microframework, although this does not mean that it is a platform limited by its simplicity. One of its main advantages is that it allows us to use virtual environments, ideal for preventing conflicts between interpreter versions and disorder of packages and repositories, enabling the creation of exclusive environments for each application without interfering with others, while the global interpreter remains clean and with the option to create more virtual environments for new applications if required (Grinberg, 2014).

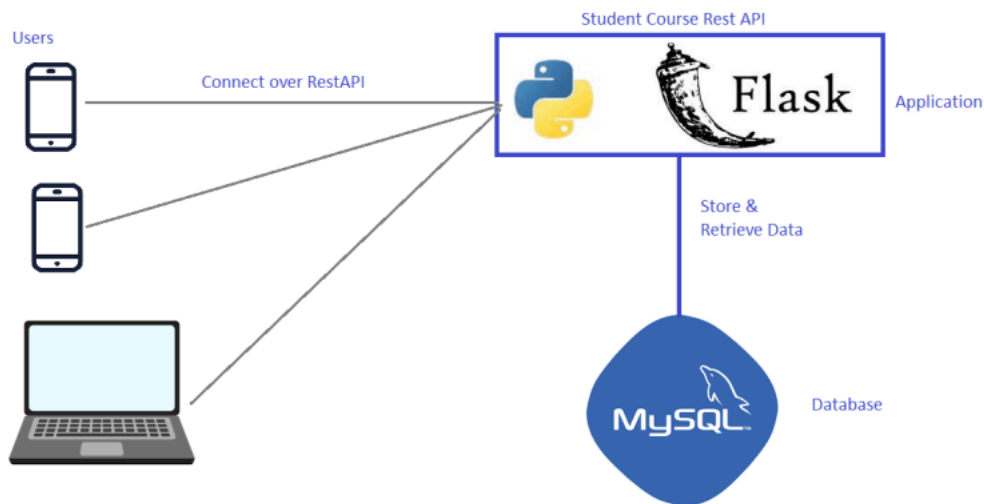


Figure 9: Example of integration using the Flask framework.

Source: Direct DevOps.blog (Available at <https://directdevops.blog/2019/11/02/deploying-the-docker-application-and-mysql-with-volume-support-into-kubernetes-from-code-to-docker-registries-like-acr-ecr-and-then-to-eks-aks/>)

Figure 9 shows an example of an integration using Flask + *Python*, *within* a global environment, where end users have access to an application that calls third-party services such as in this case MySQL, *with* Flask being the *framework* that manages a web application within a virtual environment, in this way the MySQL database will never be modified by the influence of a third party, and the Flask application works transparently to the user.

CHAPTER VI: QUALITATIVE STUDY OF THE MAIN TYPES OF TUMOR

This study is aimed at the most common types of brain tumours according to the WHO, which are Meningioma, Gliomas and Pituitary.

6.1 MENINGIOMAS

These types of tumors are usually benign and in most cases present with an encapsulated anatomy. Depending on their intracranial location, they can be very harmful and sometimes lethal. The interest of its study lies in the fact that it is the brain tumour with the highest incidence in the United States between 2002 and 2006 (Wiemels, Wrensch, & Claus, 2010). Its origin may be due to multiple causes, however, it is stated that it may be due to family history, mutations in the neurofibromatosis gene (NF2), or exposure to high levels of ionizing radiation.

Compared to other brain tumours, meningioma are relatively less studied with respect to etiological risk factors, which requires more in-depth studies since many have been discovered through observation and not through surgical removal. To date, only few epidemiological studies have had significant statistical weight to separately study the risk factors that directly impact the formation of this type of tumour.

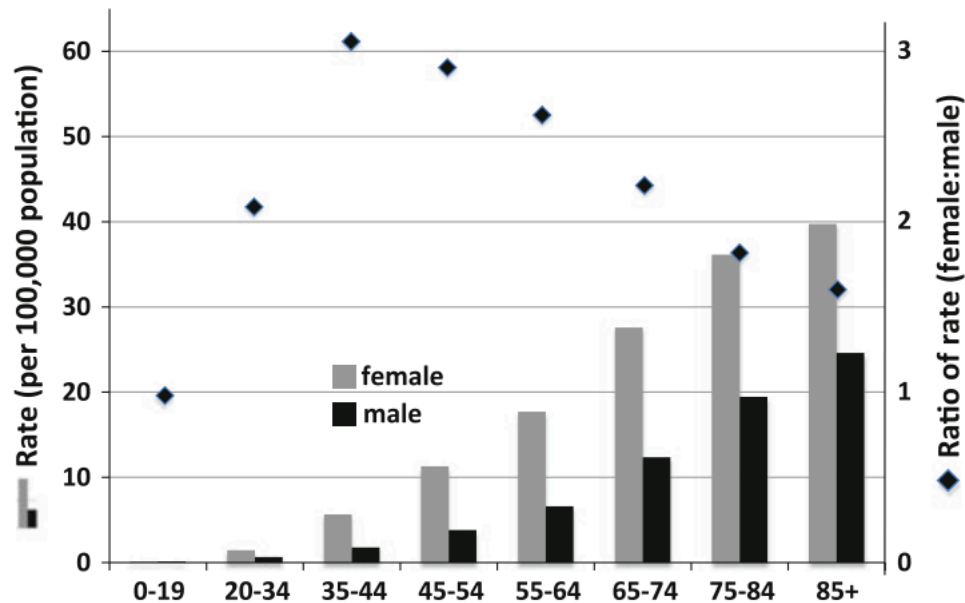


Figure 10: Incidence graph by age and sex per 100,000 inhabitants in the United States between 2002 and 2006.

Source: *Epidemiology and Etiology of Meningioma* (Available at: <https://link.springer.com/article/10.1007/s11060-010-0386-3>)

In Figure 10, we can see how the incidence of Meningioma in the older population is much more frequent and mainly in women, while in the case of younger people the incidence is considerably lower, however, it is still women who are more likely to suffer from this disease. On the other hand, from the graph it can be inferred that the population under 20 years of age has a very low or almost zero incidence, however, in cases where the disease manifests itself in children there is a high percentage of possibility that it will become lethal (USA National Cancer Institute, 2020).

With respect to the anatomy of Meningioma, they are classified according to their characteristics, molecular structure and location of the tissue, divided into three grades, with number 1 being the slowest in growth, and number 3 being the malignant anaplastic meningioma whose growth is much greater.

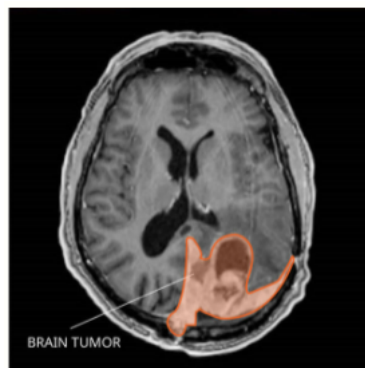


Figure 11: Magnetic resonance imaging (MRI) of a brain meningioma.

Source: USA National Cancer Institute, Gov. (Available at: <https://www.cancer.gov/rare-brain-spine-tumour/tumours/meningioma>)

Figure 11 shows an MRI image of a moderately developed Meningioma, frequently grade II and III manifest as a mass in the external layer that covers the brain tissue. Sometimes malignant meningioma can also spread to other parts of the central nervous system through the cerebrospinal fluid and even possibly reach bone tissue.

6.2 PITUITARY TUMOUR

The pituitary tumor is also shown as one of the tumors with the highest incidence in medical diagnoses; in the United States alone, in 2015, some 13,210 cases of this type of tumor were detected (Siegel, Miller, & Jemal, 2017). This type of tumour is complex in terms of its manifestation since it could be asymptomatic, or present secondary symptoms or due to endocrine dysfunction.

Some of the main symptoms directly impact the visual field of people, affecting vision. Headaches and hydrocephalus can also be mentioned in more extreme cases due to the secretion of active peptide liquid (Trincado Martínez, 2003). This tumor does not have an indefinite form since it varies according to its nature and its degree of development and growth. This type of tumor usually lodges in the region of the pituitary glands located at the base of the brain. The impact of this disease is widely dangerous because the pituitary glands produce hormones that affect the functions of other glands in the body.

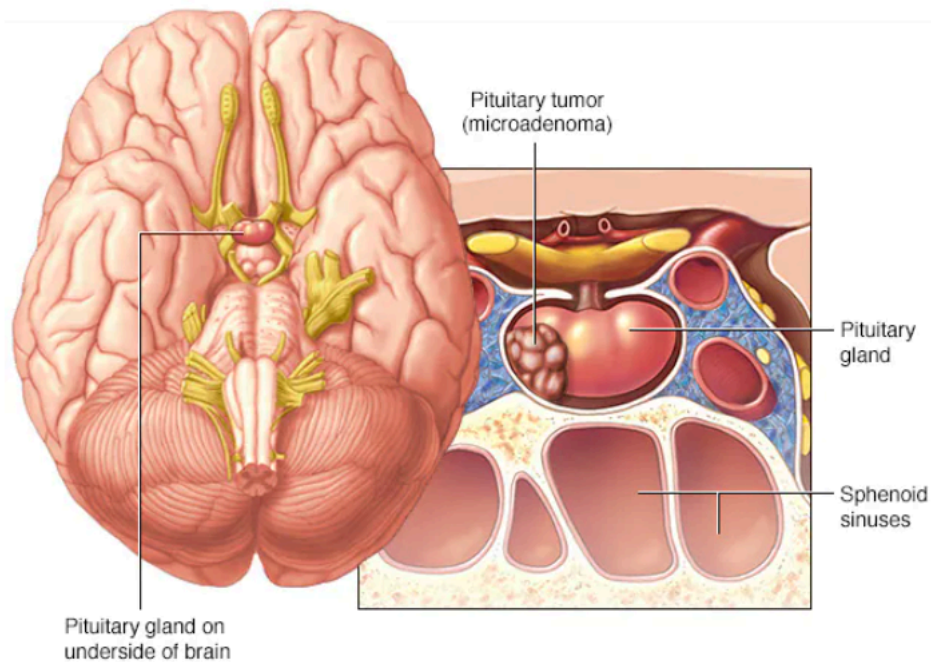


Figure 12: Superior and lateral view of pituitary microadenoma located at the base of the pituitary gland.

Source: Mayo Clinic (Available at: <https://www.mayoclinic.org/es-es/diseases-conditions/pituitary-tumours/symptoms-causes/syc-20350548#dialogId42104319>)

Figure 12 shows a pituitary tumour located at the base of the pituitary glands, manifesting as a tissue that forms and grows in an area affecting pituitary functions which are fundamental in the development of brain and endocrine activities. The following image shows what an MRI looks like with a top and side view of a pituitary tumour.

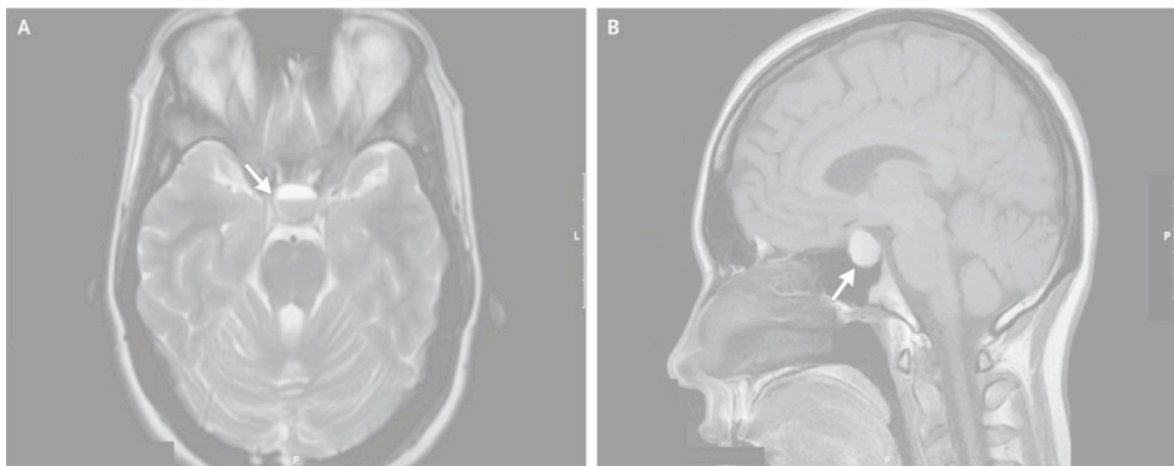


Figure 13: Images obtained by MRI of a pituitary tumor A) Top view B) Left lateral view.

Source: The New England Journal of Medicine (Available at: <https://www.nejm.org/doi/full/10.1056/nejmicm0900500>)

Figure 13 shows an MRI image of a patient with a pituitary tumour. In the box on the right we see the location of the tumour from a top view, and the box on the right shows us the same tumour from the lateral view.

6.3 GLIOMA

Another of the main tumours of the central nervous system, whose information about the cells that cause this disease is still unknown, although it is true that in-depth studies have been developed throughout history in order to specify its origin and certain hypotheses have been presented based on clinical evidence. However, as is the case with Meningiomas and pituitary tumours, it is not a simple task to generate predictive models due to their heterogeneity and variability in the way in which these tumours present.

The incidence of gliomas is approximately 5 to 10 cases per 100,000 inhabitants and they include tumors from low-grade or benign tumors to the most complex and malignant due to their physiological characteristics in size, shape, region where they are formed and time of development in the human brain, which can cause collateral damage such as headaches, mental changes, language disorders, vision alterations, endocrine dysfunction, among others (Reyes Oliveiros & Lema Bouzas, 2007).

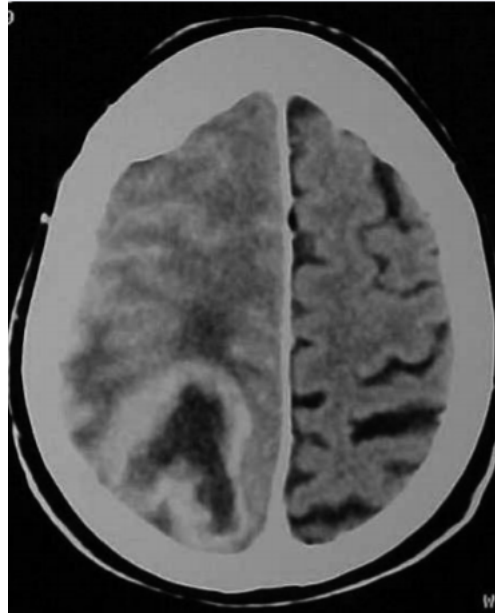


Figure 14: Computed Tomography Image of a Glioma located in the axial lobe.

Source: Neuropsychological characterization of patients with glioma treated at the Medellin Cancer Institute. (Available at: https://www.acnweb.org/acta/2008_24_1_13.pdf)

Figure 14 shows us a major grade IV glioma, which refers to a tumour of about 3 centimeters located in the axial lobe. This type of patient presents cancerous tissues in a very sensitive region of the brain whose treatment can be mixed depending on the medical diagnosis, since it may require chemotherapy, radiotherapy or surgery.

6.4 IMAGES OBTAINED BY MAGNETIC RESONANCE (MRI).

Images obtained by magnetic resonance have been used in the medical field for more than 40 years and continue to be used today for medical purposes due to the large number of advantages that this technique offers. A magnetic resonance image (MRI) is nothing more than a procedure in which a magnetic field and computer-controlled radio waves are applied to see internal tissues without having to invade any tissue, organ, bone or part of the body.

Doctor of Medicine Jaume Gili defines magnetic resonance as:

“a physical phenomenon by which certain particles such as electrons, protons and atomic nuclei with an odd number of protons (Z) and/or an odd number of neutrons (N) can selectively absorb radiofrequency energy” (Gili, 2002)

The phenomenon occurs in the presence of powerful magnetic fields, and is possible because some tissues have a physiological response when subjected to temporary radiation, which is why it is considered a non-invasive imaging technique that can facilitate high image resolutions, a fact that greatly facilitates the medical diagnosis process.

The MRIs are obtained from devices composed of two large tubular-shaped magnets so that the patient can enter inside them and once a magnetic field is emitted, the water molecules present in the body are configured in another position, then, through radio wave radiation, the tissue emits a response that, although very small, is powerful enough to create an image from said response.

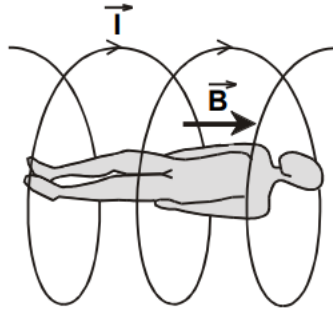


Figure 15: Physical interpretation of the magnetic field created by the generating device during the image acquisition process.

Source: Biophysical Introduction to Magnetic Resonance in Neuroimaging, page 2.1

Figure 15 shows the induced magnetic field B to which a patient is subjected during the phase of obtaining resonance, along a solenoid-shaped conductor through which an electric current I circulates. Of course, this technique can be highly dangerous if safety precautions are not followed rigorously. For this it is necessary to be free of any metallic element during the session. Reporting it to the radiology technical staff is essential.

Figure 16 shows the graphical representation of a device used to produce MRI images.

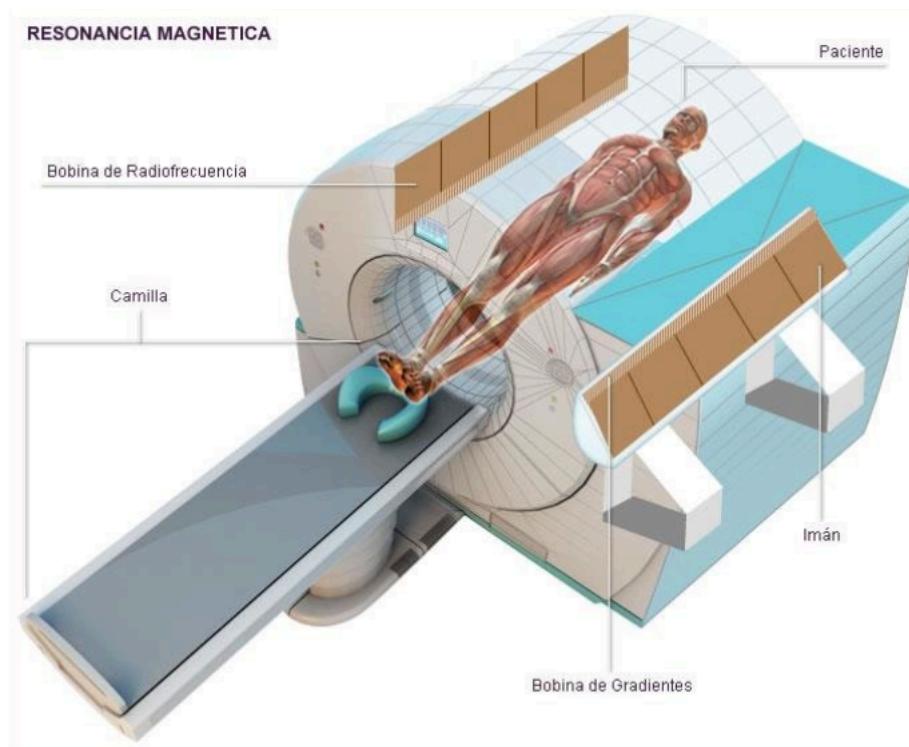


Figure 16: Graphic representation of an MRI machine

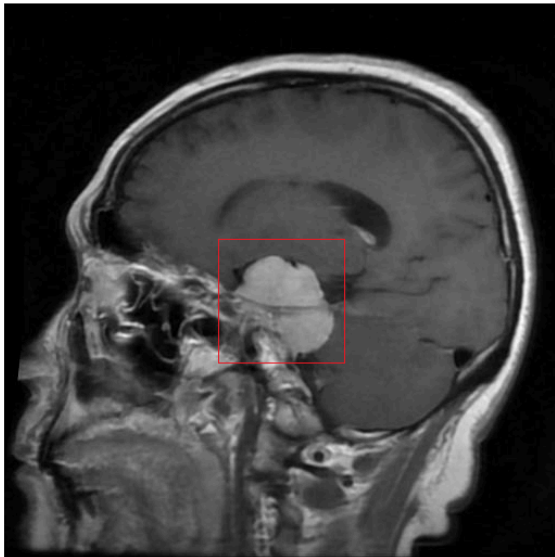
Source: ResearchGate (Available at: https://www.researchgate.net/figure/La-compra-de-resonancia-magnetica-nuclear-RMN-consiste-en-usar-un-poderoso-campo_fig4_329020406)

As we can see, the device is no longer a tubular chamber where the patient must lie down rigorously following the instructions of the radiology technicians, removing from the body any metallic device or clothing with metallic accessories, the coils act as magnets, when an electric current is applied they generate a magnetic field that travels in a known direction, by interacting with the radio waves that cause the body's tissues to become sensitized, this is how the MRI image is obtained.

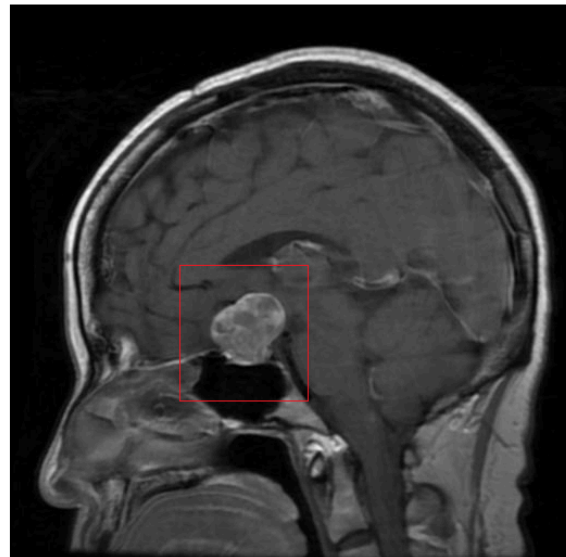
6.5 NATURE OF A BRAIN TUMOUR

Reference was previously made to the particularity of brain tumours in that they are characterized by being widely heterogeneous; there are no defined patterns that determine their shape, size or location within the brain tissue. This represents the main challenge of this research, since two different tumours can have a very similar location, size and shape, therefore, training a neural network effectively is not an easy task.

Next, we see in Figure 17 a real case of two tumours (Meningioma and Pituitary), with a very similar anatomy, the size and shape is very similar in both cases, the location point is also very similar, both images belong to different patients.



MENINGLIOMA



PITUITARIO

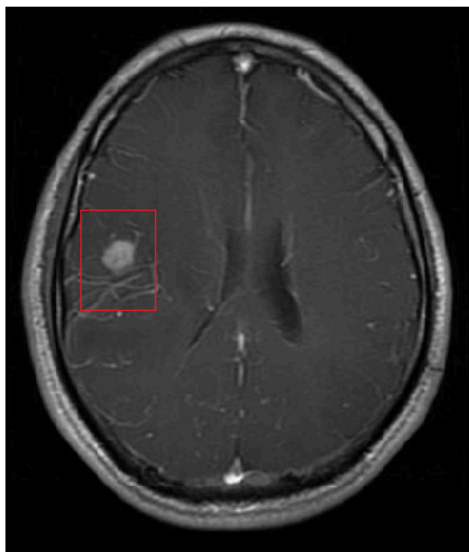
Figure 17: Anatomical similarity of a Meningioma and Pituitary tumor.

Source: Figshare Dataset (Available at: https://figshare.com/articles/dataset/brain_tumour_dataset/1512427)

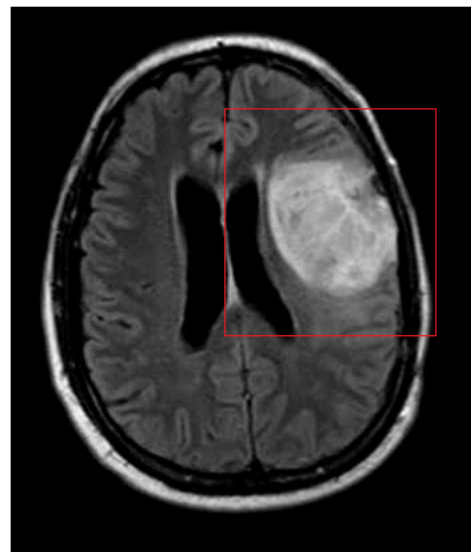
On the other hand, we know that tumours can present symptoms from early stages, or, there is the possibility that the patient manifests symptoms until advanced stages of the tumour, this demonstrates the complexity of making an effective prognosis from the first time.

1.

GLIOMAS



Fase temprana



Fase avanzada

2.

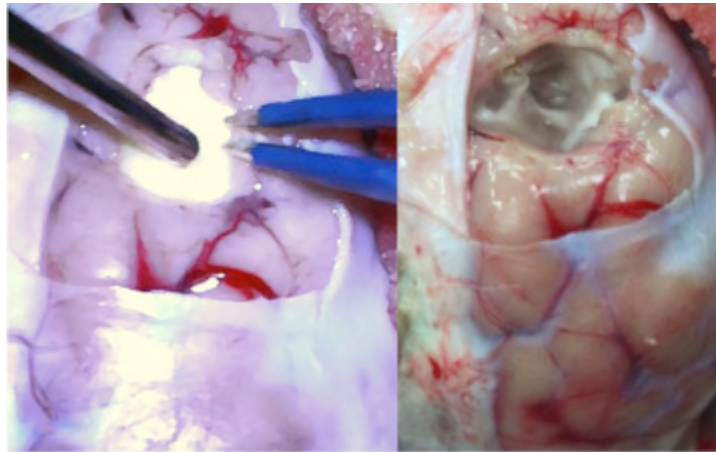


Figure 18: a) Early and Advanced phase of a Glioma brain tumour. b) Real image of the removal process of an advanced Glioma tumor in a patient

Source: Figshare Dataset (Available at: https://figshare.com/articles/dataset/brain_tumour_dataset/1512427)

Figure 18a demonstrates the difference in sizes that the same tumour can represent at different stages of development. It is evident that a tumour in an advanced stage has a high chance of causing irreversible damage to the patient or becoming lethal. Hence arises the interest on the part of the medical community to be able to detect this disease in early stages, since the detection time can save the life of a person suffering from this disease.

On the other hand, Figure 18b shows the process of removing an advanced Glioma tumour in a patient, through which a neurosurgeon processes the membrane that protects the brain to gain access to the cancerous tissue. These surgeries are usually very complex due to the area where the cancer is located, so a high level of knowledge and experience in practice is required.

CHAPTER VII: SYSTEM DEVELOPMENT AND IMPLEMENTATION

The implementation of the system is based on a design based on *Big Data* through the use of *datasets* that allow us to train the network efficiently. The more input images we use to perform the training, the better the result and performance will be. This work shows the use of different previously trained CNNs in order to show which one provides better results in the execution, in the same way a self-created model will be generated in the same way to be able to compare the effects on the output. As mentioned in previous sections, the project aims to propose the use of different techniques that allow contrasting the different experimental results.

7.1 DATASETS USED IN IMPLEMENTATION

The datasets represent one of the most important parameters with which our neural network will be developed, since, as mentioned above, they allow us to provide input data to our network so that it can recognize images according to their characteristics. The main challenge of this development is justified by the fact that brain tumours usually have similar shapes, sizes and also location within the brain area in similar regions, so we must have a CNN model that is sufficiently precise to be able to differentiate between one tumour and another.

For this research development, the following datasets freely available on the web have been used, which will be detailed below:

7.1.1 IMPROVED PERFORMANCE DATASET

- Enhanced Performance of Brain Tumour Classification via Tumor Region Augmentation and Partition (Cheng, et al., 2015)

Available at: https://figshare.com/articles/dataset/brain_tumour_dataset/1512427

The structure of this dataset groups tumours according to the types of tissue that make up the tumour and is called the Region of Interest (ROI) tissue type and focuses on three types of tumour: Meningioma, Glioma and Pituitary obtained by magnetic resonance imaging, which, by the way, is the most popular and conventional technique in the diagnosis of brain tumours.

7.1.2 DATASET KAGGLE

- Classification of brain tumors (MRI) (Chakrabarty & Kanchan, 2020).

Available at: <https://www.kaggle.com/sartajbhuvaji/brain-tumour-classification-mri>

Kaggle is a free access web platform designed for the creation and exchange of data sets to train neural networks. This allows the development community to have access to more than 50,000 datasets with more than 400,000 notebooks to create, improve, experiment and test Machine Learning and Deep learning projects (Kaggle, 2010).

7.1.3 GITHUB DATASETS

- GitHub Brain Tumour Detector (Ali Habib, 2019)

Available at: <https://github.com/MohamedAliHabib/Brain-Tumor-Detection>

Another free access medium that allows projects to be uploaded to its platform for exchange and interaction between the developer community, among its main advantages we can mention not only the fact of being open source, but also that it allows files, datasets, repositories, among other things, to be uploaded to the cloud, necessary for the implementation of projects.

7.2 DATASET PREPARATION

As indicated previously, the datasets are of utmost importance for the correct training of our neural network. The efficiency with which our neural network model will work depends on this dataset, among other things. The greater the efficiency, the higher the percentage of reliability we will tend to have in the prediction results. On this occasion, Google Colab has been used using the Google Drive interface, it is in Google Drive that we will store our collection of images, which for our purposes will be images. Next, we see in the following image the way we have distributed our dataset.

My Drive > Colab Notebooks > BT_Dataset ▾

Name ↑	Owner	Last modified
 Test	me	Aug 2, 2021
 Train	me	Jun 29, 2021
 Validation	me	Jun 29, 2021

Figure 19: folder distribution in Google Drive.

Source: Google Colab (Own creation)

As can be seen in Figure 19, the dataset has been divided into 3 different folders: Train, Validation and Test, so that 70% of the total images have been used for training, 20% validation and 10% for testing. Within each of the folders there are each of the classes that correspond to the types of tumour that you want to predict as follows:

My Drive > Colab Notebooks > BT_Dataset > Test ▾

Name ↑	Owner	Last modified
 Meninglioma	me	Aug 2, 2021
 Pituitary	me	Aug 3, 2021

Figure 20: Internal structure of each of the folders for Training, Validation and Testing.

Source: Google Colab (Own creation)

For this work we will do a combination of datasets and we will see the result in the implementation, the works that have been carried out with these datasets normally do not usually combine the datasets, but on this occasion we will do a purification of the datasets where we will eliminate the low resolution images and we will combine them to see the effect on the output.

7.3 OVERFITTING

The 70-20-10 rule that we have used in our network is not definitive for all cases and all CNNs, this is just a random choice, however, some bibliographies suggest the distribution of the datasets in *one way* or another to avoid "overfitting", a term used in statistics to describe when a model contains too much information about a sample or class to the point that it is only able to learn data from that class but when presented with other data it is not able to recognize it since it does not have enough information about that class. class. As Pablo Argibay has cited in the report on Advanced Statistics: the problem of overfitting and the method of minimum descriptions:

"The paradox of overfitting is defined as: complex models contain more information about the sample data, but less information about the future data (prediction)" (Argibay, 2011)

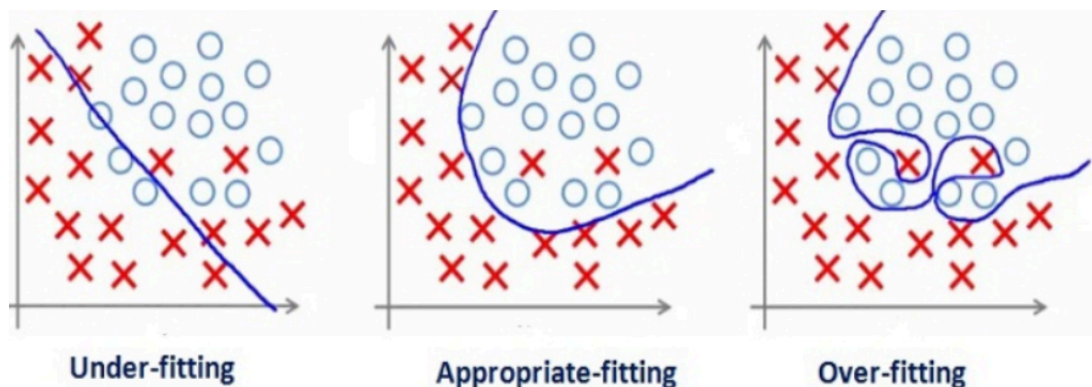


Figure 21: Behavior of an overfitting function.

Source: Dream Learning (Available at: <https://dreamlearning.ai/neural-network-from-scratch-elementos-indispensables-teoria-optimizadores-inicializados-regularizadores-y-otros-hyperparametros-parte-11/>)

As we can see in Figure 21, the problem of overfitting is addressed, this is mainly due to the algorithm or function receiving too much data from one or more classes, that is, when overtraining occurs, therefore, when it must recognize other patterns or characteristics in the image, the system will not be able to recognize them, the ideal would be to have a function that is capable of correctly classifying the different classes. On the other hand, in the image on the left we also have the case of "underfitting" which we also have to avoid, since it also gives us an inaccurate reading at the output.

The case of underfitting occurs when the algorithm has not been able to effectively learn each of the different classes. This may be caused by the quality of the images in the datasets or because we have not introduced a sufficient amount of input data for the algorithm to be able to efficiently learn these classes. As in the case of overfitting, underfitting could generate false positives or false negatives in the reading of the output of our system.

The ideal would be to try to make the function similar to the image we see in the center of the figure, in which we see how the function is capable of discriminating the different classes efficiently, although while it is true, reaching 100% effectiveness is really a complex task, we can have very close readings, the closer to 100%, the better our prediction will be.

7.4 USE OF PRE-TRAINED CNNs AND TRANSFER LEARNING

The use of pre-trained neural networks is a really useful action for the purposes of training our model. It must be taken into account that there are previous works that have demonstrated through reproducible results that it is possible to obtain a level of effectiveness close to 97%, models that are quite accurate when compared to the first developments of CNN. Some of the main pre-trained CNNs that will be used in this work will be discussed later. For now, it will only be necessary to say that the use of the main CNN architectures known as VGG16 and Inception can be implemented to reproduce the results and be able to make a comparative analysis of their advantages and disadvantages.

With the passage of time, CNNs are increasingly used in different fields of science and technology, and this has meant that a higher level of precision is required in the systems that use this technique, which is why fine-tuning was introduced, in which highly accurate neural networks are used that have gone through an immensely meticulous training process, using huge databases, which allows the weights of the layers of the neural network to be adjusted, so that this model can be used in other networks. (Kang, Cho, Yoon, Park, & Kim, 2021).

This facilitates a much faster and more precise training process, providing better use of time, as well as increasing the reliability of the network. These models perform the task of classification and segmentation in much more heterogeneous image classes, as is our case with the Brain Tumour datasets.

7.5 THE CONVOLUTION

Convolution in a neural network is a technique widely used in image processing to extract features from an image. This method is mainly used for classification and segmentation. The term convolution was introduced for the first time in 1980 by Kunihiko Fukushima. However, 18 years later this method was improved by introducing a new approach through “backward programming” or backpropagation in which the sum of the errors in each of the layers of the network is used. neuronal by calculating the descending gradient to later compare it with the desired output. This method reproduces automatically and allows the neural network to recognize features or patterns in an image with a defined pixel size (Wu, 2017).

Convolution in an image of size A x B pixels is carried out by applying filters that are essentially smaller in size than the image itself, Figure 22 shows us this effect.

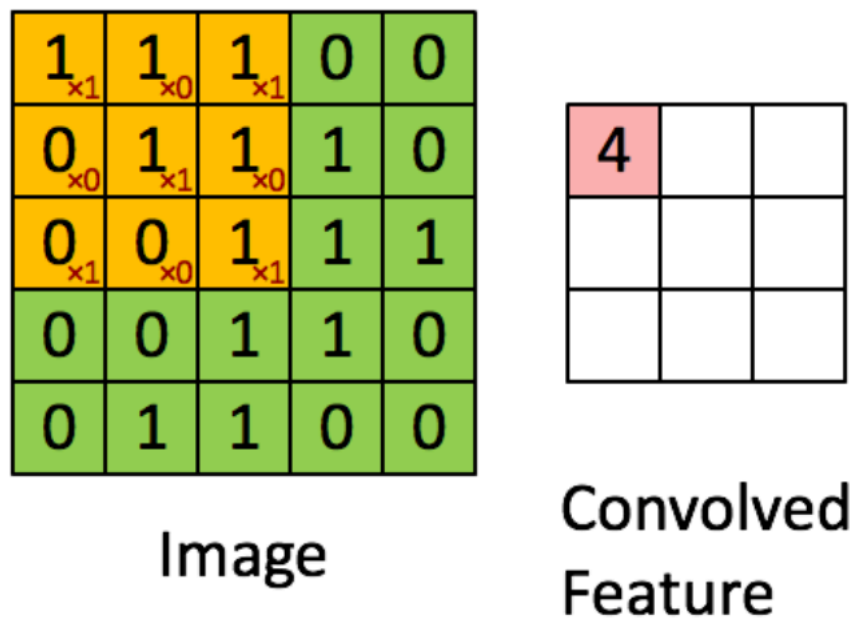


Figure 22: Convolution applying a 3 x 3 pixel filter for a 5 x 5 pixel image.

Source: Ice Cream Labs. (Available at: <https://medium.com/@icecreamlabs/3x3-convolution-filters-a-popular-choice-75ab1c8b4da8>)

In it we can see how the application of the filter gives us a new output in a smaller image, this representation shows a single-dimensional image, normally the images are 3-dimensional for each of the RGB channels, this allows us to extract some features in each of the layers of our network.

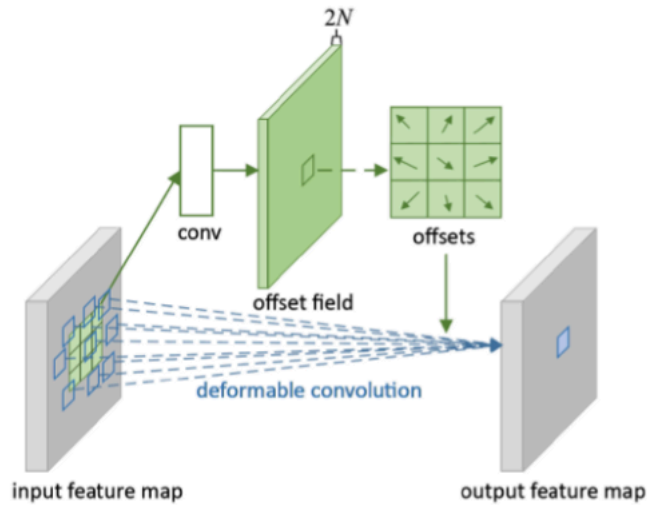


Figure 23: Performance of a convolution on an image

Source: Archana Blog. (Available at: <https://archana1998.github.io/post/rahul-sukthankar/>)

In Figure 23, we see the effect of the convolution applied to an image, the filter that we mentioned above is applied to a certain segment of an image, and it is this action that allows us to extract the characteristics, an action that will subsequently allow us to recognize which category the image belongs to, and in this way how the classification or segmentation in images is carried out. This technique is valid for applying it to video sequences since they are made up of several images per second. Of course, a very high computational resource is required to be able to process the algorithm, but the same effect that is produced in the output is essentially the same.

7.6 VGG16

VGG16 is a convolutional neural network, which has been trained on a subset of ImageNet, which uses a collection of more than 14 million images belonging to more than 20 thousand different classes, achieving results with more than 92% effectiveness in image classification (Tammina, 2019). This architecture is widely used for the transfer learning technique and its structure is seen below in Figure 24:

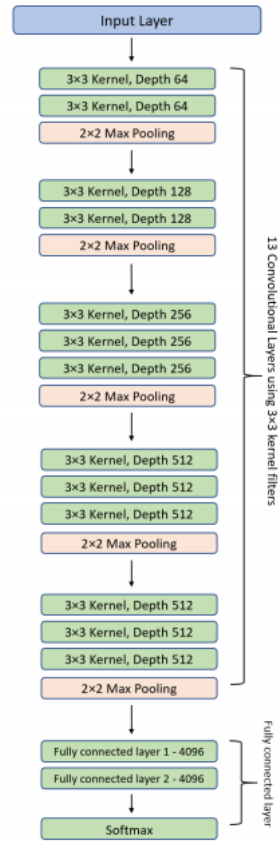


Figure 24: Architecture of the VGG16 model with 2 fully connected layers.

Source: International Journal of Scientific and Research Publications (Available at: [Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images \(ijssrp.org\)](http://www.ijssrp.org)).

The structure has two convolutional layers at the input, with a 64 depth filter with a 3 x 3 kernel, followed by 2 layers with a 128 depth filter with a 3 x 3 kernel, then it has 2 layers with a 256 depth filter with a 3 x 3 kernel, then 6 layers with a 512 depth filter with a 3 x 3 kernel separated into 3 layers in one block and 3 in another, finally two fully connected layers with an output module with softmax activation, each of the modules has a 2 x 2 MaxPooling activation layer.

Layer	Patch size	Input size
conv*2	3*3/1	3*224*224
pool	2*2	64*224*224
conv*2	3*3/1	64*112*112
pool	2*2	128*112*112
conv*3	3*3/1	128*56*56
pool	2*2	256*56*56
conv*3	3*3/1	256*28*28
pool	2*2	512*28*28
conv*3	3*3/1	512*14*14
pool	2*2	512*14*14
fc	25088*4096	25088
fc	4096*4096	4096

Figure 25: Summary table of the VGG16 architecture.

Source: International Journal of Scientific and Research Publications (Available at: [Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images \(ijssrp.org\)](http://www.ijssrp.org)).

Figure 25 shows us the aforementioned, the standard size of the image is 224 x 224 px, so we must adjust and preprocess the image before subjecting it to the training and validation process of the neural network.

7.7 INTEGRATION OF THE GOOGLE COLAB ENVIRONMENT

As mentioned in sections 5.1 and 5.2, the work environment that we have chosen to develop the algorithm is Google Colab, for which you only need to have a Google account (it can be Gmail or any other Google service). One of the great advantages of using this service is that the integration is simple. Below is the process to be able to use the datasets that we have stored in Google Drive in our Google Colab development environment. The following Figure 26 shows how to start the process of enabling Google Drive in the Colab environment.

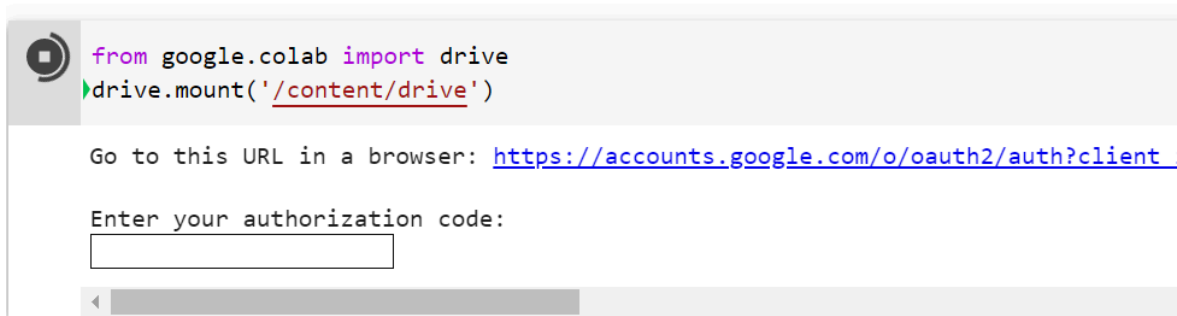


Figure 26: Google Drive integration with the Google Colab environment.

Source: Google Colab (Own creation)

By entering the integration command that is documented in the official Google Colab user guide (Google Inc., 2018), an authorization key is enabled that is accessed through a link that is generated automatically. Once said key has been entered, we can start using the Google Colab environment with the Jupyter Notebook interface (See section 5.3).

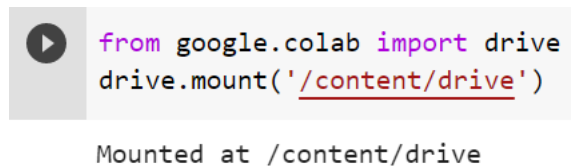


Figure 27: Integration message successfully completed.

Source: Google Colab (Own creation)

In Figure 27, we see how the assembly of Google Drive is satisfactorily done, from this moment we can use the datasets and information stored in our drive, which is precisely where we have our dataset stored, so that we have access to each of the folders where the images that we use as a reference to train our network are stored, these images come in presentations of different sizes and resolutions, since they are mixed datasets that have been collected from different databases (See section 7.1), therefore they are not homogeneous images, and must be subsequently treated to standardize the size and in this way the algorithm can act on them with a common pattern.

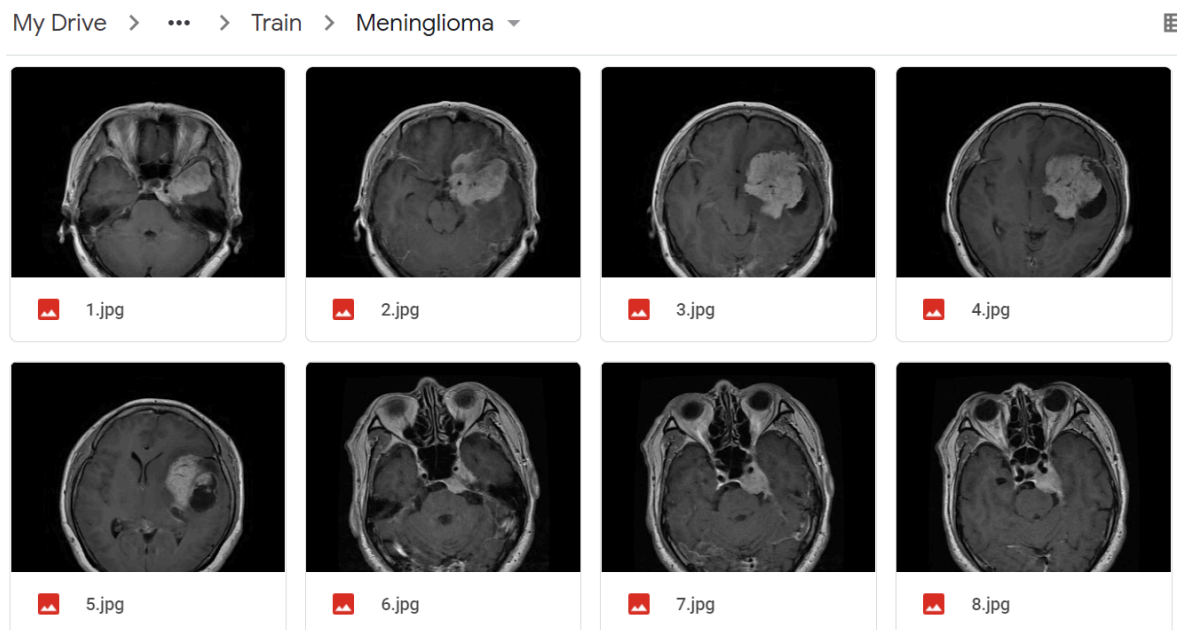


Figure 28: Example of storing datasets for training, tumor type Meningioma.

Source: Google Colab (Own creation)

Figure 28 shows a real example of storing images in a folder that is precisely our dataset. In this particular case, each image is in .JPG format because it is a significantly light format so that we can store a significant amount of images in our Google Drive.

CHAPTER VIII: EXPERIMENTAL RESULTS

This chapter will discuss the results obtained during the execution of the algorithms created for the classification of the different types of brain tumours, which were explained in detail in section 6. Next, the main particularities of the development process will be explained, as well as the effect that occurs on the output.

8.1 USE OF GRAPHIC PROCESSING UNIT (GPU)

In section 5.1 it was indicated that one of the main advantages of using *Google Colab* is that it provides us with a GPU for free, which is used to increase the graphic processing capacity in the processing of images and videos, this represents an important characteristic for our purposes of our implementation.

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
NVIDIA-SMI		470.57.02		Driver Version: 460.32.03			CUDA Version: 11.2		
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
GPU	Name	Persistence-M		Bus-Id		Disp.A	Volatile Uncorr. ECC		
Fan	Temp	Perf	Pwr:Usage/Cap		Memory-Usage		GPU-Util	Compute M.	MIG M.
=====									
0	Tesla T4		Off		00000000:00:04.0 Off				0
N/A	33C	P8	9W / 70W		0MiB / 15109MiB		0%	Default	N/A
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
Processes:									
GPU	GI	CI	PID	Type	Process name			GPU Memory	
	ID	ID						Usage	
=====									
No running processes found									
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									

Figure 29: GPU information provided by Google Colab.

Source: Google Colab (Own creation)

As we see in Figure 29, Google Colab is assigning a Tesla T4 GPU from NVIDIA, this unit was created for the purposes of implementing AI projects, offering high performance for image processing, videos and everything related to computational graphics processing, understanding that modern technological demands demand high processing capacity and quality. We can consult the technical specifications in the following image (NVIDIA, 2019).

GPU Architecture	NVIDIA Turing
NVIDIA Turing Tensor Cores	320
NVIDIA CUDA® Cores	2,560
Single-Precision	8.1 TFLOPS
Mixed-Precision (FP16/FP32)	65 TFLOPS
INT8	130 TOPS
INT4	260 TOPS
GPU Memory	16 GB GDDR6 300 GB/sec
ECC	Yes
Interconnect Bandwidth	32 GB/sec
System Interface	x16 PCIe Gen3
Form Factor	Low-Profile PCIe
Thermal Solution	Passive
Compute APIs	CUDA, NVIDIA TensorRT™, ONNX

Source: NVIDIA Official. (Available at: <https://www.nvidia.com/en-us/data-center/tesla-t4/>)

```
physical_devices = tf.config.experimental.list_physical_devices('GPU')
print("GPUs Available: ", len(physical_devices))
tf.config.experimental.set_memory_growth(physical_devices[0], True)
```

GPUs Available: 1

Source: Google Colab (Own creation)

```

Tue Aug 24 14:27:17 2021
+-----+
| NVIDIA-SMI 470.57.02    Driver Version: 460.32.03    CUDA Version: 11.2     |
+-----+-----+-----+-----+-----+-----+
| GPU   Name               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
| 0 Tesla K80              Off          | 00000000:00:04:0 Off |                    0 |
| N/A   71C    P8      33W / 149W   | 0MiB / 11441MiB |      0%    Default  |
|                                           MIG M.           |
+-----+-----+-----+-----+-----+-----+
+
Processes:
+-----+-----+-----+-----+-----+-----+
| GPU   GI    CI          PID    Type    Process name                      GPU Memory |
| ID   ID                                 Name                                Usage     |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+

```

On this occasion, *Google Colab* has assigned us a Tesla K80 model GPU as can be seen in Figure 32, *GPU* manufactured by *NVIDIA* like the Tesla T4, both very similar in terms of processing capacity, the most important differences lie in their price since the Tesla K80 model GPU is much older than the T4, therefore, they were built with different architectures.

8.2 INITIAL PARAMETERS

Before starting the training phase of the neural network, some extremely relevant parameters must be defined since these must determine the treatment that we will give to our neural network. Some of the main parameters that we must establish are:

- Image size: On this occasion we will standardize the size of each of the images to 224 x 224 pixels since each of these images that come from various *datasets* obtained from different websites (Refer to section 7.1) have different sizes and resolutions.
- Number of images that will be used in sampling: In our case we will initially use a sampling or batch size of 16. The batch size is the number of images on which the algorithm will act in each of the learning epochs and has a very important correlation with the learning rate according to studies such as the one demonstrated in “The effect of batch size on the generalization of convolutional neural networks in a histopathology dataset” (Ibrahim & Castelli, 2020). In Figure 33 the following results can be seen.

Test AUC		
Batch size	Adam LR = 0.0001	Adam LR = 0.001
16	0.9677	0.9144
32	0.9636	0.9332
64	0.9616	0.9381
128	0.9567	0.9432
256	0.9585	0.9652

Figure 33: Effect on learning performance using the ADAM optimizer for different batch sizes.

Source: ScienceDirect (Available at:

https://www.researchgate.net/publication/337816564_Analysis_of_various_optimizers_on_deep_convolutional_neural_network_model_in_the_application_of_hyperspectral_remote_sensi

- Classes: We will carry out the initial tests with 2 classes to be able to adjust the parameters. Once satisfactory results are achieved, we will proceed to add more classes to corroborate the performance and thus be able to find out if the product is the same.

8.3 USE OF LIBRARIES

For the development of this work, the libraries offered by TensorFlow and Keras will be used. This will allow us to develop the main functions of our application. Each of these libraries has a special competence to develop one or another operation within the development of the code.

```
[2] import numpy as np
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.models import load_model

from tensorflow.keras.layers import Activation, Dense, Flatten, BatchNormalization, Conv2D, MaxPool2D, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import categorical_crossentropy
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from sklearn.metrics import confusion_matrix

import itertools
import os
import shutil
import random
import glob
import matplotlib.pyplot as plt
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
%matplotlib inline
```

Figure 34: Use of TensorFlow and Keras libraries for system development.

Source: Google Colab (Own creation)

As Figure 34 shows, we can see that the following libraries are imported:

1. TensorFlow: Essential for the neural network's own functions, its layers, activation, optimizers and graphing functions.
2. NumPy: Used for algebraic and numerical calculations that involve the main operations that will be developed.
3. Itertools: Necessary to create loop interactions within the Python environment.
4. OS: Which allows integration with the operating system of the computers with which the system is developed.
5. Shutil: Offers high-level functions for managing the files you will work with.
6. Random: It has to do with numerical management and its distribution in the different battery configurations.
7. Glob: A small but necessary library to be able to search and manage files in the system, when they are heterogeneous in terms of file type, extension, etc.
8. Matplotlib: For generating graphs from obtained data.
9. Warning: To display error and warning messages when an execution failure occurs, or when there is a threat of possible damage to a particular file.

8.4 INITIAL TESTS

To fully enter into practice, we have chosen a *small* dataset with only two classes to be able to determine the effect on the implementation of our first neural network, for this we have chosen a small portion of data from *the* Kaggle set (see section 7.1.2), in which we have chosen 440 images for training, 120 images for validation and 60 for testing. The classes we will work with on this occasion are Meningioma and Non-Tumor. When executing the code we can confirm that the choice of images is correct, as shown in *Figure 35*.

```
Found 840 images belonging to 2 classes.
Found 240 images belonging to 2 classes.
Found 120 images belonging to 2 classes.
```

Figure 35: Total images chosen for Training, Validation and Test, for both classes.

Source: Google Colab (Own creation)

Indeed, we check the total number of images in the previous figure, where we have balanced datasets by total number of images. The number of images used in the folders dedicated to training, validation and testing should not necessarily be the same for each class, this is a random decision that in our case we have designated it this way so that there are no differences in the learning rates that we will apply later.

Next, we proceed to apply a filter that is provided by the Keras and TensorFlow libraries, which helps the algorithm to better identify and highlight the most outstanding characteristics of the images in our dataset, with the sole purpose that our neural network can make a correct classification based on its characteristics. The following figure shows us the application of this filter on each image.

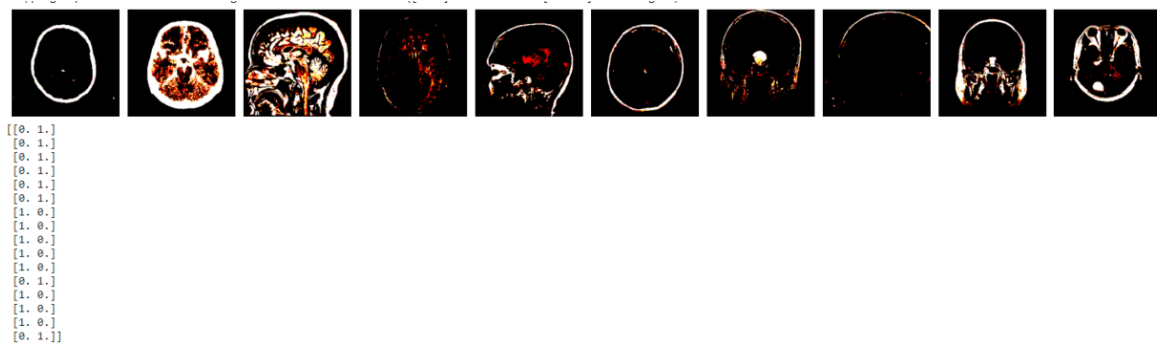


Figure 36: Application of RGB filter to extract image features with class prediction.

Source: Google Colab (Own creation)

Figure 36 shows us the application of the filter to extract the main characteristics of the image. This allows the algorithm to recognize the patterns that will subsequently allow for a more effective learning execution. It is important to note that the images before this step came with different resolutions and sizes. By performing this action, we homogenize the image to standardize the input of our neural network.

On the other hand, the arrays that we see under the images represent which class each of the images belongs to, in the form:

Meningioma [0. 1.], in binary classification: 01

Non-tumour: [1. 0.], in binary classification: 00

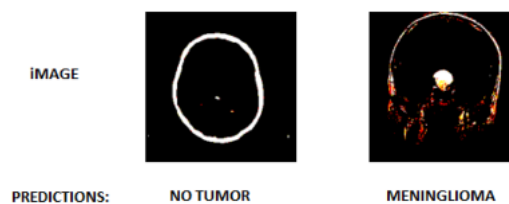


Figure 37: Image classification according to class.

Source: Google Colab (Own creation)

In Figure 37 we can better detail how the application of the filter allows us to better observe the presence or absence of a brain, the image on the left clearly exemplifies a healthy brain while the image on the right shows a brain with a tumor, this action is called pre-processing, and there are different modes and filters to apply at this stage, the official Keras documentation tells us that we can apply any of the following modalities: Grayscale, RGB and RGBA, if the image is going to be converted into 1, 3 or 4 channels (Keras Doc, 2015).

On this occasion we will use a model of our own development, which we summarize in the following table.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 224, 224, 64)	1792
conv2d_11 (Conv2D)	(None, 224, 224, 64)	36928
max_pooling2d_4 (MaxPooling2D)	(None, 112, 112, 64)	0
conv2d_12 (Conv2D)	(None, 112, 112, 128)	73856
conv2d_13 (Conv2D)	(None, 112, 112, 128)	147584
max_pooling2d_5 (MaxPooling2D)	(None, 56, 56, 128)	0
conv2d_14 (Conv2D)	(None, 56, 56, 256)	295168
conv2d_15 (Conv2D)	(None, 56, 56, 256)	590880
max_pooling2d_6 (MaxPooling2D)	(None, 28, 28, 256)	0
conv2d_16 (Conv2D)	(None, 28, 28, 512)	1180160
conv2d_17 (Conv2D)	(None, 28, 28, 512)	2359808
max_pooling2d_7 (MaxPooling2D)	(None, 14, 14, 512)	0
Flatten_1 (Flatten)	(None, 100352)	0
dense_3 (Dense)	(None, 4096)	411045888
dense_4 (Dense)	(None, 4096)	16781312
dense_5 (Dense)	(None, 2)	8194

Total params: 432,520,770
 Trainable params: 432,520,770
 Non-trainable params: 0

Figure 38: Summary of the model created in the initial test.

Source: Google Colab (Own creation)

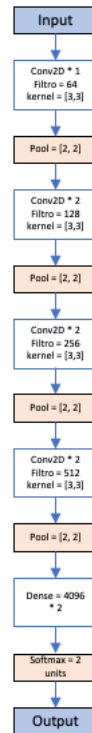


Figure 39: Block diagram of created model.

Source: Own elaboration.

As we see in Figure 38 and Figure 39, the model that we have created generates a total of approximately 432 Million trainable parameters, distributed in the way expressed in Figure 30, where we apply the concept of Conv2D and Pool that we detail below:

- Conv2D: This type of layer is available in the Keras and TensorFlow libraries, it creates a bias vector that produces a tensor at the output of said layer, as the official Keras documentation says, when used in the first layer it provides an argument called input_shape that defines the shape of the input and the channels.
- MaxPooling2D: An equally two-dimensional layer that downsamples the input along its spatial dimensions, taking the maximum value over an input window. Likewise, pooling creates a new resulting output which allows the extraction of features from an image as mentioned in section 7.5.
- Activation by layers: Each of the layers of a neural network must produce an output that, in turn, will be the input of the next layer. The most used activation functions in CNN are:

1. Sigmoid: Whose output values are between 1 and 0, and are defined by the equation:

Equation 3

$$\sigma(z) = 1 / (1 + e^{-z})$$

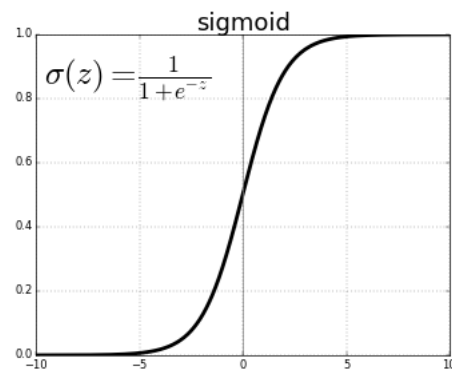


Figure 40: Sigmoid activation function*

1. *ReLU*: Which produces a positive active output for any positive input.

Equation 4

$$f(x) = \max(0, x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

* Source: ml4a.github.com (Available at https://ml4a.github.io/ml4a/es/neural_networks/)

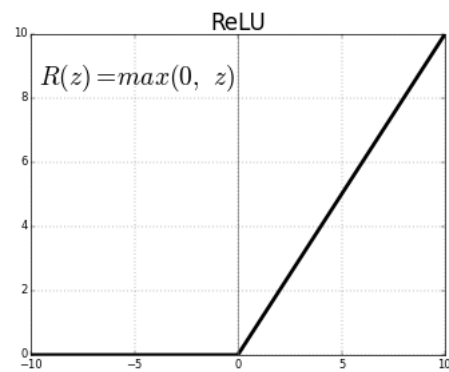


Figure 41: ReLU activation function**

1. *Softmax*: When the total sum of all possible outputs is equal to 1.

Equation 5

$$f(z_j) = e^{z_j} / \sum_{k=1}^K e^{z_k}$$

* Source: ml4a.github.com (Available at https://ml4a.github.io/ml4a/es/neural_networks/)

** Source: ml4a.github.com (Available at https://ml4a.github.io/ml4a/es/neural_networks/)

1. *tanh*: comprises values strictly less than 1 and strictly greater than -1.

Equation 6

$$\tanh(x) = 2 / (1 + e^{-2x}) - 1$$

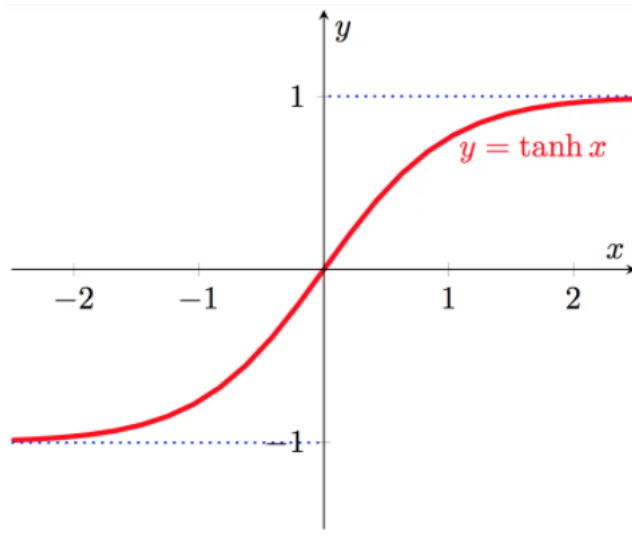


Figure 42: Tanh activation function.

Source: JournalDev (Available at: <https://www.journaldev.com/49083/tanh-activation-function>)

The last layer of our neural network model has a softmax activation function *due* to its logistic regression generalization property, which will make a calculation of all the probabilities of each class within a total of possibilities.

When proceeding to train the system we realized that the model is not exact, and has an unstable behavior pattern during the training process. The explanation for this phenomenon is overfitting. As we have previously pointed out, the dataset we are using is very small and is not sufficient taking into account that MRI (Magnetic Resonance) images are very heterogeneous due to the great diversity of shapes, sizes and aspects of brain tumours, and in this case meningliomas.

Next, Figure 43 shows the accuracy in the validation process and the losses obtained during the training process, for this we have chosen 15 epochs.

```
117/117 - 21s - loss: 0.0020 - accuracy: 1.0000 - val_loss: 0.0715 - val_accuracy: 0.9722
Epoch 48/50
117/117 - 21s - loss: 0.0019 - accuracy: 1.0000 - val_loss: 0.0673 - val_accuracy: 0.9741
Epoch 49/50
117/117 - 21s - loss: 0.0018 - accuracy: 1.0000 - val_loss: 0.0664 - val_accuracy: 0.9741
Epoch 50/50
117/117 - 21s - loss: 0.0016 - accuracy: 1.0000 - val_loss: 0.0561 - val_accuracy: 0.9815
```

Figure 43: Accuracy in validation and behavior of losses during the training process.

Source: Google Colab (Own creation)

The accuracy in the validation process is quite acceptable, however, as we observed, it is not the behavior we expect, since we assume an increasing accuracy each time we pass a training epoch.

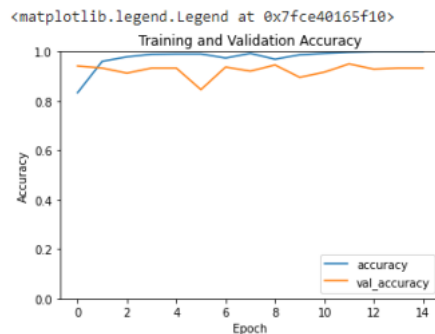


Figure 44: Precision graph in the training and validation process.

Source: Google Colab (Own creation)

Figure 44 shows us a very good performance of the algorithm during the training process, reaching 100% accuracy. We have a similar case during the validation process where results close to 94% accuracy are obtained, not bad taking into account that we are working with a relatively small dataset.

On the other hand, we have the graph that represents the losses for training and *validation* in Figure 45, as we can see in the following figure, the behavior during training is as expected, during the first epochs the loss is clearly large, however, as the epochs progress, the losses decrease, until they reach values close to zero.

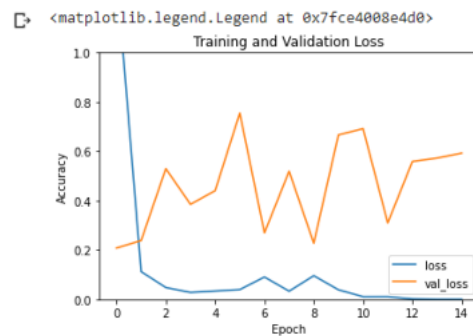


Figure 45: Loss graph during the training and validation process.

Source: Google Colab (Own creation)

One of the functions that the Keras and TensorFlow library offers us is the so-called confusion matrix, which allows us to see the results obtained more clearly in a more visual way. In our case, we apply this function to our Test dataset. Through the confusion matrix we can notice the number of images that it detects correctly and falsely.

The most appropriate interpretation that we could give to *the confusion matrix* is a linear arrangement of all the possibilities that an image may have to be correctly or incorrectly classified.

Matriz de confusión para una variable de salida dicotómica (clasificación)		Resultado de la predicción	
		Sí	No
Valor real de la clase	Sí	Verdadero positivo	Falso Negativo
	No	Falso positivo	Verdadero negativo

Figure 46: Arrangement of a confusion matrix

Source: MicroMedix (Available at: <https://micromedix.me/2018/01/27/cardiopatas-en-las-cosas-del-corazon-no-basta-con-una-segunda-opinion/matriz-de-confusion/>)

In Figure 46, we can see the arrangement of a confusion matrix, where the upper left and lower right quadrants show us the correct predictions, whether true positive or true negative for a real value belonging to a class, while the upper right and lower left quadrants show us the predictions not correct by the system, whether they are false positive or false negative. This visual aid is very useful since it facilitates the interpretation of the predictions of the Test images, and it also helps us know in a simple and direct way whether the system is accurate or imprecise.

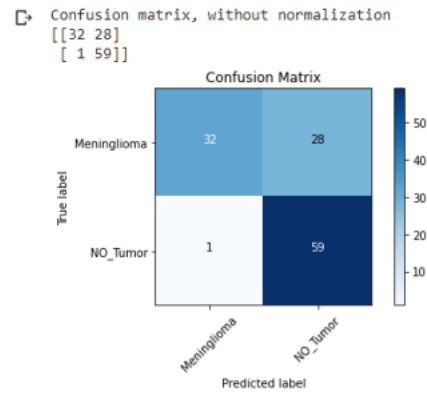


Figure 47: Confusion matrix for the Meningioma and NO_Tumor classes.

Source: Google Colab (Own creation)

Figure 47 shows the predictions produced by the neural network. For the NO_Tumor class, only 1.66% of the test images were classified incorrectly. By contrast, 46.66% of the Meningioma predictions were incorrect - almost half of the test images in that class - which makes this initial model insufficiently reliable for practical use.

8.5 IMPROVED TRAINING PROCESS

As we have seen, working with small datasets usually produces underfitting during the training and validation process, therefore, the results are prone to be affected in the output since the system is not able to “learn” effectively either in one or in several classes, therefore, when we introduce a new image to the neural network so that it can classify it based on the true predictions, the results are moderately or highly inaccurate. For this reason, we will now introduce a larger dataset with the same classes as the previous example (See section 8.3) and see the effect.

The architecture of our network is essentially the same that we have applied in section 8.4, and that we have implemented for feature extraction in each of its layers. Below, Figure 49 shows us each of the layers and blocks that form our neural network. This format is available in the Keras library and we obtain it through the matplotlib library as shown in Figure 48.

```
[ ] from keras.utils.vis_utils import plot_model
    plot_model(model, to_file='model_plot.png', show_shapes=True, show_layer_names=True)
```

Figure 48: Matplotlib function to obtain block neural network model.

Source: Google Colab (Own creation)

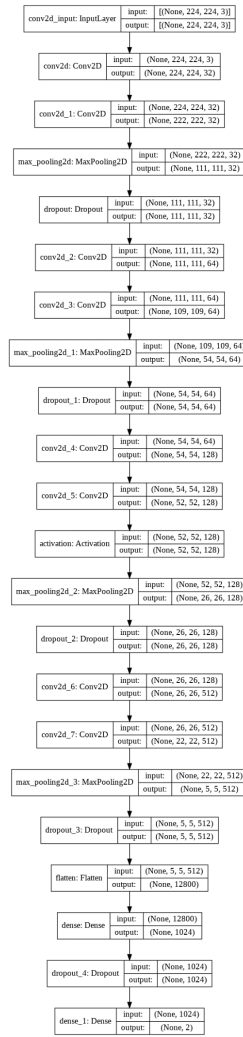


Figure49

Source: Google Colab (Own creation)

Another aspect to take into account is that we will apply an *adjustment* to the datasets, since in the previous example a small dataset was used (See Figure 35) where the results were very poor. On this occasion we will use a larger dataset, and in the same way we will do a classification of images to eliminate those with low resolution, since they can produce a poor training and learning process, which is precisely what should be avoided.

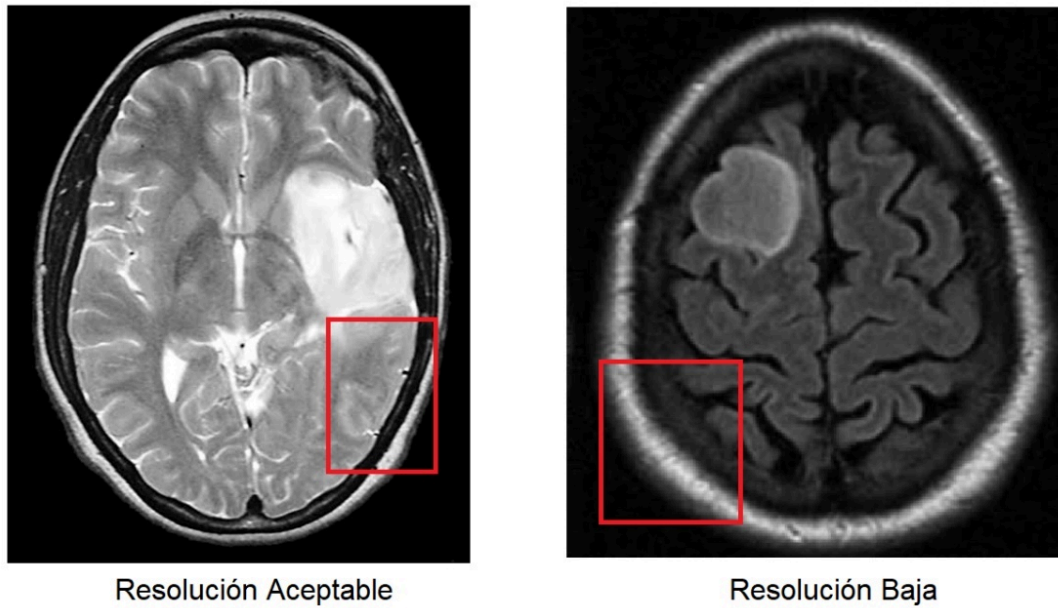


Figure 50: MRI of Gliomas with different resolutions.

Source: Figshare Dataset (Available at: https://figshare.com/articles/dataset/brain_tumour_dataset/1512427)

Figure 48 shows us the resolution differences that we can find in the *dataset*, the quality in both images is notably unequal and when we introduce these images to our neural network to start the training and validation process, we produce poor learning of the neural network, causing the results to have very low reliability and therefore insufficient performance.

The image on the left is obviously much superior in terms of quality and resolution, and its counterpart, the image on the right, is the opposite. If we look closely at the section in the red box, we see a pronounced blurring in the perimeter margins that can cause confusion in the feature extraction process. On the other hand, it must be taken into account that the format we are using is the JPG format.

Using this type of format has the advantage that they are lightweight files, therefore, they are easier to manipulate and store on disk or in the cloud, but they have the great disadvantage of having a low resolution, and although neural networks are designed taking these variables into account, there is always *the risk of* producing under-fitting or poor training due to image quality. Once the datasets have been *purified* by eliminating images with poor resolution, we proceed to organize the folders for Training, Validation and Test again, and execute the code.

Another aspect that should be mentioned is that before starting the training process we are going to pre-process the images so that the system can train the network in a better way and thus have an easier time extracting its features.

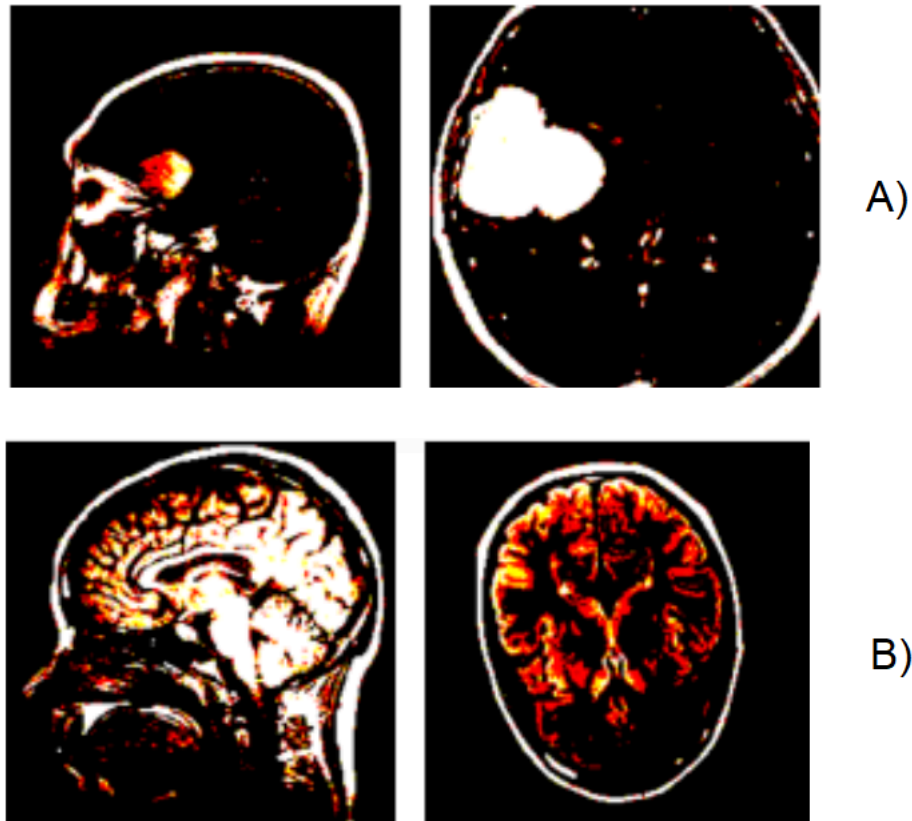


Figure 51: Pre-processed images before starting the training of the neural network. A) Brain with tumor in lateral and frontal view. B) Brain without presence of tumors in lateral and frontal view.

Source: Google Colab (Own creation)

Figure 52 shows us the number of images that we will use in this next test, the classes are the same as the ones we were working with in the previous test, Meningioma and NO_Tumor. This time we will use 1860 images for Training (930 for each class), 540 images for Validation (270 for each class) 260 images for Test (130 for each class).

```
Found 1860 images belonging to 2 classes.
Found 540 images belonging to 2 classes.
Found 260 images belonging to 2 classes.
```

Figure 52: Experimentation with new dataset.

Source: Google Colab (Own creation)

Another technique that we will *apply is data augmentation*, which we will talk about later. For now, we will only say that it allows the modification of images from their original form to a new modified configuration, in which we can rotate it, enlarge or reduce it, rotate the original direction, among other options in order to be *able* to expand the number of images in the dataset to train our neural network. Next, we can observe the training and validation process by training our neural network with these new conditions. Likewise, we have increased the number of epochs to see improved loss and learning *graphs*. Figure 53 shows these results.

```

Epoch 2/30
117/117 - 27s - loss: 0.4335 - accuracy: 0.8226 - val_loss: 0.5794 - val_accuracy: 0.6481
Epoch 3/30
117/117 - 27s - loss: 0.3271 - accuracy: 0.8720 - val_loss: 0.2963 - val_accuracy: 0.8778
Epoch 4/30
117/117 - 27s - loss: 0.2261 - accuracy: 0.9194 - val_loss: 0.2344 - val_accuracy: 0.9222
Epoch 5/30
117/117 - 27s - loss: 0.1595 - accuracy: 0.9435 - val_loss: 0.2100 - val_accuracy: 0.9241
Epoch 6/30
117/117 - 27s - loss: 0.1433 - accuracy: 0.9452 - val_loss: 0.2957 - val_accuracy: 0.8704
Epoch 7/30
117/117 - 27s - loss: 0.1047 - accuracy: 0.9656 - val_loss: 0.3452 - val_accuracy: 0.8741
Epoch 8/30
117/117 - 27s - loss: 0.0534 - accuracy: 0.9855 - val_loss: 0.3685 - val_accuracy: 0.9185
Epoch 9/30
117/117 - 27s - loss: 0.0681 - accuracy: 0.9796 - val_loss: 0.2661 - val_accuracy: 0.9148
Epoch 10/30
117/117 - 27s - loss: 0.0650 - accuracy: 0.9790 - val_loss: 0.3440 - val_accuracy: 0.9093
Epoch 11/30
117/117 - 27s - loss: 0.0521 - accuracy: 0.9833 - val_loss: 0.2053 - val_accuracy: 0.9259
Epoch 12/30
117/117 - 27s - loss: 0.0234 - accuracy: 0.9925 - val_loss: 0.1645 - val_accuracy: 0.9481
Epoch 13/30
117/117 - 27s - loss: 0.0255 - accuracy: 0.9919 - val_loss: 0.3113 - val_accuracy: 0.9241
Epoch 14/30
117/117 - 27s - loss: 0.0462 - accuracy: 0.9833 - val_loss: 0.7274 - val_accuracy: 0.8685
Epoch 15/30
117/117 - 27s - loss: 0.0687 - accuracy: 0.9774 - val_loss: 0.1751 - val_accuracy: 0.9389
Epoch 16/30
117/117 - 27s - loss: 0.0204 - accuracy: 0.9935 - val_loss: 0.3382 - val_accuracy: 0.9333
Epoch 17/30
117/117 - 27s - loss: 0.0311 - accuracy: 0.9914 - val_loss: 0.1647 - val_accuracy: 0.9519
Epoch 18/30
117/117 - 27s - loss: 0.0159 - accuracy: 0.9962 - val_loss: 0.2137 - val_accuracy: 0.9352
Epoch 19/30
117/117 - 27s - loss: 0.0118 - accuracy: 0.9941 - val_loss: 0.2748 - val_accuracy: 0.9426
Epoch 20/30
117/117 - 27s - loss: 0.0068 - accuracy: 0.9968 - val_loss: 0.5863 - val_accuracy: 0.8889
Epoch 21/30
117/117 - 27s - loss: 0.0359 - accuracy: 0.9882 - val_loss: 0.3489 - val_accuracy: 0.9333
Epoch 22/30
117/117 - 27s - loss: 0.0063 - accuracy: 0.9984 - val_loss: 0.3994 - val_accuracy: 0.9278
Epoch 23/30
117/117 - 27s - loss: 0.0105 - accuracy: 0.9957 - val_loss: 0.5951 - val_accuracy: 0.9074
Epoch 24/30
117/117 - 27s - loss: 0.0152 - accuracy: 0.9941 - val_loss: 0.4436 - val_accuracy: 0.9204
Epoch 25/30
117/117 - 27s - loss: 0.0293 - accuracy: 0.9903 - val_loss: 0.1770 - val_accuracy: 0.9426
Epoch 26/30
117/117 - 27s - loss: 0.0130 - accuracy: 0.9952 - val_loss: 0.1697 - val_accuracy: 0.9648
Epoch 27/30
117/117 - 27s - loss: 0.0098 - accuracy: 0.9973 - val_loss: 0.2219 - val_accuracy: 0.9463
Epoch 28/30
117/117 - 27s - loss: 0.0035 - accuracy: 0.9989 - val_loss: 0.2932 - val_accuracy: 0.9519
Epoch 29/30
117/117 - 27s - loss: 0.0074 - accuracy: 0.9968 - val_loss: 0.2596 - val_accuracy: 0.9259
Epoch 30/30
117/117 - 27s - loss: 0.0890 - accuracy: 0.9747 - val_loss: 0.1904 - val_accuracy: 0.9333

```

Figure 53: Summary of the model created in the second test with a larger dataset.

Source: Google Colab (Own creation)

Again, we will do the tests with the images that we have designated for Test, we have designated the same number of images of both classes to be able to make the predictions under equal conditions, again the classes we have are: Meningioma and NO_Tumor with the labels designated for each class, below, they are shown in Figure 54.

```

test_batches.class_indices
{'Meningioma': 0, 'NO_Tumor': 1}

```

Figure 54: Neural network class labels.

Source: Google Colab (Own creation)

We see the distribution of the images with their respective labels in the Test folder in the image below in Figure 55.

```
[ ] test_batches.classes
```

[illegible]

Figure 55: Distribution of images by class.

Source: Google Colab (Own creation)

Unlike the first test, the results are now notably better in terms of accuracy in the training and validation processes, likewise the losses produced have decreased and the graphs have improved, *Figure 53* shows us how the final results have made the system now have a very high reliability when making the predictions, now the confusion matrix will show us visually and numerically the results of said predictions.

☞ Confusion matrix, without normalization

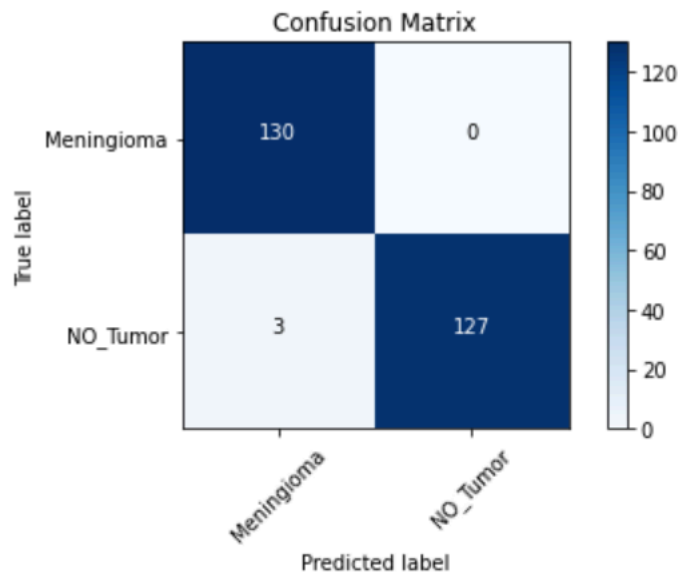
$$\begin{bmatrix} 130 & 0 \\ 3 & 127 \end{bmatrix}$$


Figure 56: Confusion matrix for the Meningioma and NO_Tumor classes with new dataset.

Source: Google Colab (Own creation).

As shown in Figure 54, the *accuracy* of the prediction made on the Meningioma class is **100%** accurate, while for NO_Tumor it is **2.3%**, that is, taking into account the nature of the images with which we are working, as well as the possible deficiencies in terms of resolution and quality of these in the *dataset*, we can ensure that we have a highly reliable system.

The loss and accuracy graphs in the training and validation processes demonstrate that very satisfactory results were obtained, which indicates that the increase in the number of images in the dataset plays an essential role in our neural network and directly impacts the quality of the predictions and results.

```
[ ] <matplotlib.legend.Legend at 0x7f8cc4ff26d0>
```

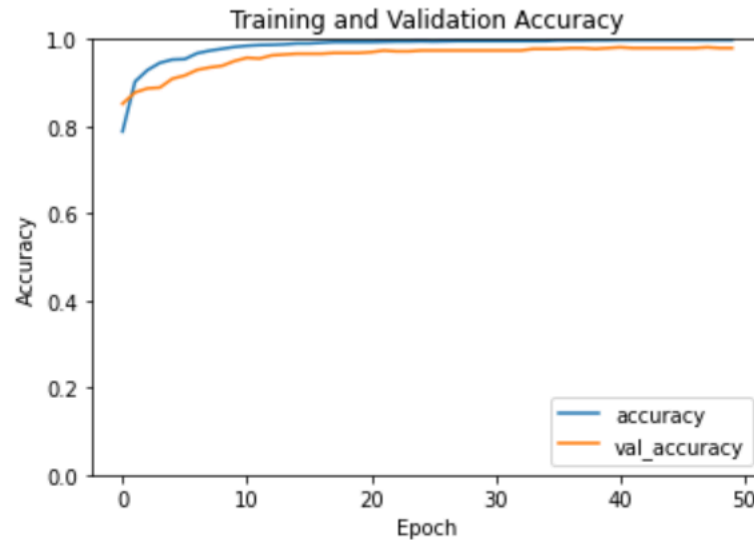


Figure 57: Precision graph of the training and validation processes.

Source: Google Colab (Own creation).

Figure 57 shows us the way in which the neural network learns progressively in each interaction or epoch, the output is exactly what is expected since after each epoch the learning rate becomes more precise and the loss becomes smaller, while Figure 58 shows us the losses for training and validation, which become smaller after each interaction, this clearly demonstrates that the neural network is adjusted correctly and that the learning has been done correctly, the physical difference between the images of the two classes with which we are working. It makes the output of the system even more accurate, since there is an important difference between a brain with a tumour and one that does not have it.

However, these results cannot be taken into account as a real parameter for predictions, because the network is trained with a database of few images, this generates an unreliable prediction since there is possibly *under-fitting* and the effect on the output may be biased.

```
<matplotlib.legend.Legend at 0x7f8cc4fc04d0>
```

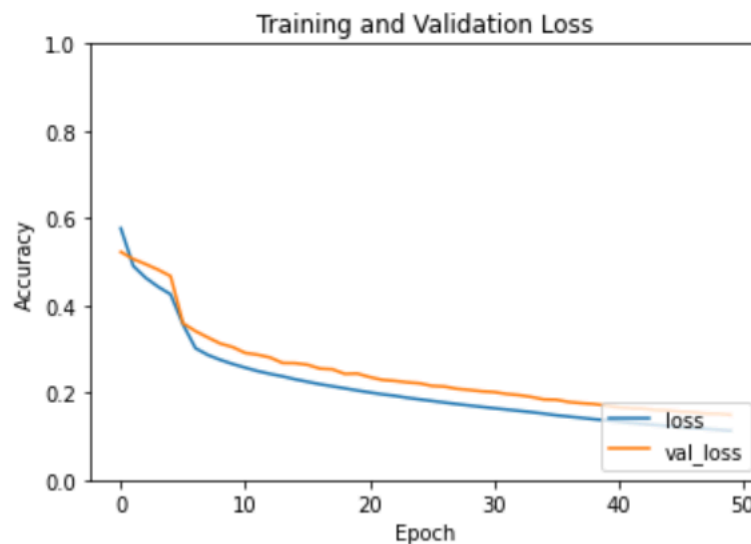


Figure 58: Loss graph of the training and validation processes.

Source: Google Colab (Own creation).

8.6 DATA AUGMENTATION

As mentioned above, the “data augmentation” technique has come to provide important help to those who work with neural models, since datasets are often the main problem or limitation to building a precise, functional and exact neural network. However, one must be extremely careful with the modifications made to the images, since the essential characteristics of the image must be preserved without distorting them to the point that we cause the reverse effect, that is, over-fitting, which is precisely what we want to avoid (Wong, Gatt, Stamatescu, & McDonnell, 2016).

Data *augmentation* allows us not only to increase the number of images *in a* dataset, but also allows us to modify the image organically, giving us the opportunity to modify many variables to avoid confusing the training process. Some of the main characteristics that allow us to modify this function are:

- **rotation_range:** Allows you to rotate the image to the right or left about its center.
- **width_shift_range:** This variable enables the image width shift range.
- **height_shift_range:** This variable enables the image height shift range.
- **shear_range:** This option allows us to modify the angle (in degrees) of cutting counterclockwise.
- **zoom_range:** Variable that allows us to zoom in on the image.
- **channel_shift_range:** It is a range for random color channel shifts.
- **horizontal_flip:** To randomly flip the image horizontally.
- **validation_split:** To randomly rotate the image vertically.

It is important to highlight that we *have applied the* data augmentation function on the images designated for training, it should never be applied on the images assigned for Test, below is the configuration that we have chosen for our network in *Figure 59*:

```
train_generator = ImageDataGenerator(rotation_range=15,
                                     width_shift_range=0.1,
                                     height_shift_range=0.1,
                                     shear_range=0.20,
                                     zoom_range=0.1,
                                     channel_shift_range=10,
                                     horizontal_flip=True,
                                     preprocessing_function=preprocess_input)
```

Figure 59: Configuration of the data augmentation function of our cnn.

Source: Google Colab (Own creation).

By applying *the data augmentation* we will obtain an output like the one we *see in* *Figure 60*, where we use the original image (marked in the red box) and modify the parameters that we saw in *Figure 59* to obtain the output that is nothing more than the original image with changes in its position, zoom, cut, etc.

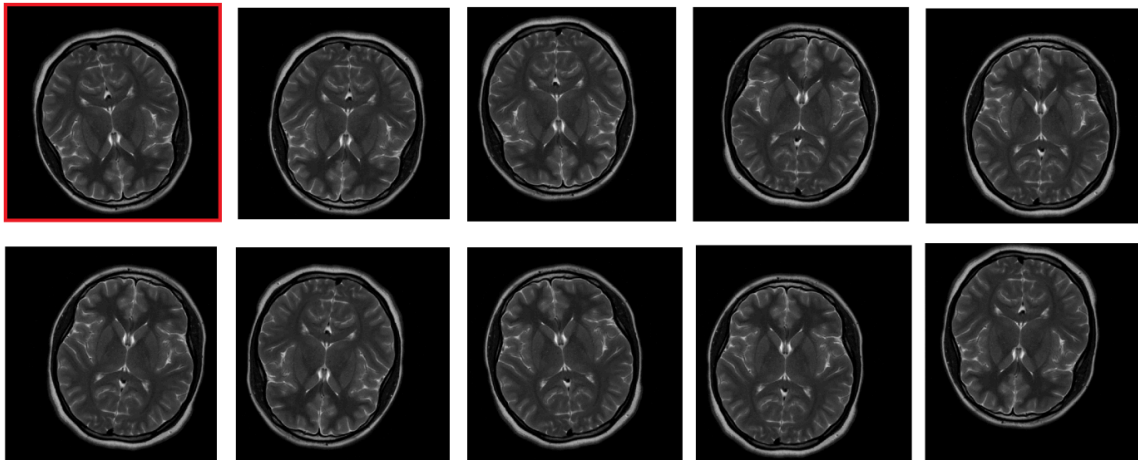


Figure 60: Result of applying data augmentation on an image.

Source: Google Colab (Own creation).

8.7 USE OF TRANSFER LEARNING

In Section 7.4 the topic of Transfer Learning was discussed, where it is ensured that this technique is applied in supervised machine learning scenarios where an attempt is made to use the prior knowledge of a neural network that has been trained with thousands of classes and images, demonstrating accuracy and highly precise performance, which allows us to save a considerable amount of time in the training process of our cnn.

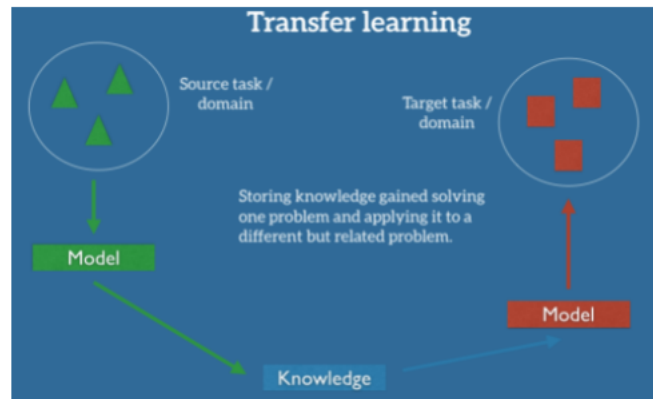


Figure 61: Transfer Learning model applied to new model.

Source: Reder.io (Available at: <https://ruder.io/transfer-learning/index.html#whatistransferlearning>)

Figure 61 demonstrates what was previously said, where the knowledge obtained previously applied in another model is used in a new model, and where the classes, datasets and type of problem are completely different, however, the weights of the layers in the neural network are stored to reproduce them in a new problem.

On this occasion, we will use the network called VGG16 that was explained in Section 7.6, where we will add the *Gioma* and *Pituitary* classes that were discussed in Chapter 6, for this we have chosen the same number of images for Training, Validation and Test, Figure 62 shows us the distribution of the datasets according to the number of classes:

```
Found 3720 images belonging to 4 classes.
Found 1080 images belonging to 4 classes.
Found 520 images belonging to 4 classes.
```

Figure 62: Dataset of 4 classes for implementation with Transfer Learning.

Source: Google Colab (Own creation).

- **Training:** 3720 images (930 for each class)
- **Validation:** 1080 images (270 for each class)
- **Test:** 520 images (130 for each class)

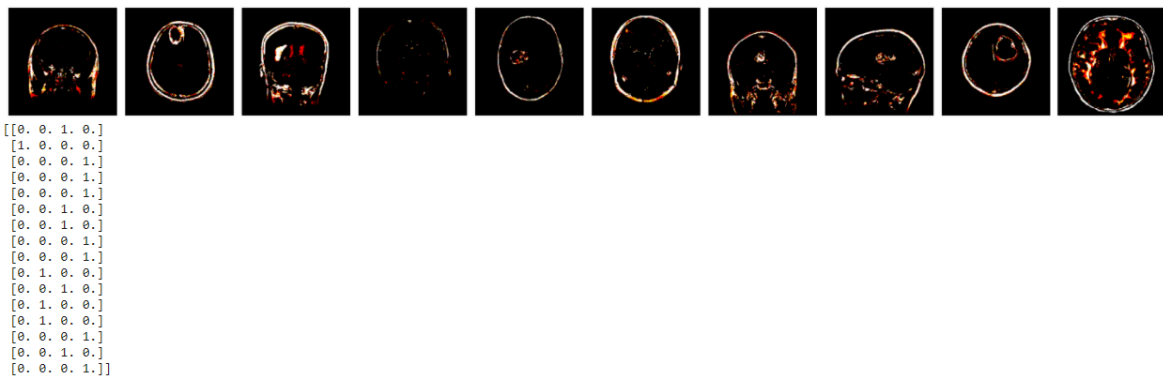


Figure 63: Labeling by classes for Transfer Learning

Source: Google Colab (Own creation).

In Figure 63, we see the labels that are assigned to each class, this time we have 4 possibilities in the prediction as follows:

- **0.0.0.1:** Meningioma
- **0.0.1.0:** NO_Tumor
- **0.1.0.0:** Pituitary
- **1.0.0.0:** Glioma

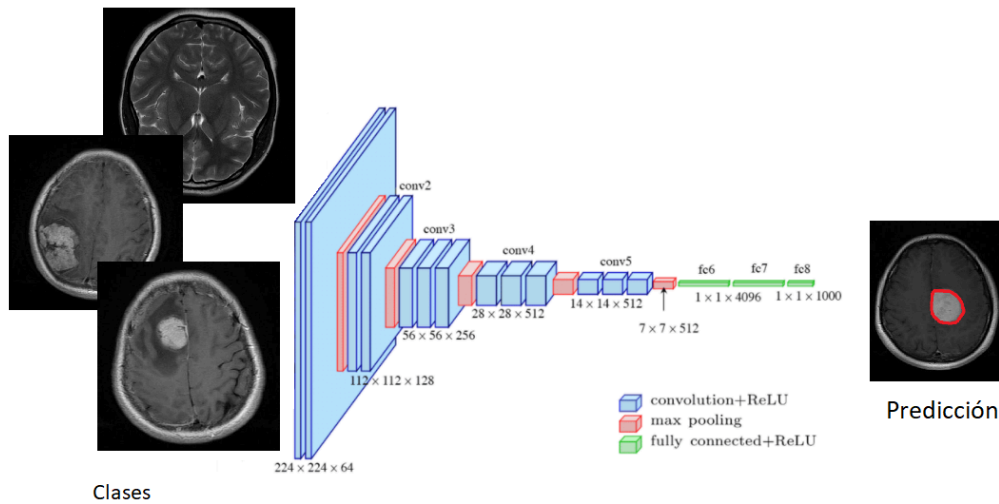


Figure 64: VGG model used for Transfer Learning.

Source: Own elaboration

Figure 64 shows us the model that we will implement on this occasion, which is the VGG16 neural network, which receives at the input the different images of 4 different classes, they pass through the input layer in a format already adjusted to the form (224, 224, 3) to standardize its size, then a max pooling is applied to give way to the next layer, in which a filter of (112, 112, 128) is applied, then After passing through two convolution layers, a *new max pooling* filter is applied to enter the next layer of dimensions (56, 56, 256). Subsequently, after applying a filter (of size 28, 28, 512), 3 subsequent layers of convolution and ReLU activation come into action. Next, the layers of (14, 14, 512) appear after the max *pooling* filter is applied. Finally, we give way to the last 3 dense layers of dimension (1, 1, 4096), after passing through a filter of (7, 7, 512), these last three layers reduce the dimensionality to 1 because this point is where the image features are extracted, thanks to this we can perform an appropriate class classification.

On this occasion, when using the Transfer Learning technique, we originally downloaded the model that is available at the following link:

https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels.h5

Once the model is downloaded we will see that the model has 3 Dense-type layers as *shown in* Figure 65, this model uses the last layer for predictions, however, it is open to adjust the number of classes in our model.

fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
predictions (Dense)	(None, 1000)	4097000
=====		
Total params: 138,357,544		
Trainable params: 138,357,544		
Non-trainable params: 0		

Figure 65: Last 3 layers of the VGG model16

Source: Google Colab (Own creation).

The total parameters that can be trained are approximately 140 million parameters, so we will adjust the last output layer so that it recognizes the 4 classes of our CNN.

fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
dense (Dense)	(None, 4)	16388
=====		
Total params: 134,276,932		
Trainable params: 16,388		
Non-trainable params: 134,260,544		

Figure 66: Last 3 layers of the VGG16 model with output layer adjustment.

Source: Google Colab (Own creation).

Now we only have 16 thousand “trainable” parameters since we have limited the system to only the 4 classes of interest. The accuracy of the system can be seen in the following Figure 67:

```

27/27 - 9s - loss: 0.0088 - accuracy: 1.0000 - val_loss: 0.3645 - val_accuracy: 0.8667
Epoch 46/50
27/27 - 9s - loss: 0.0086 - accuracy: 1.0000 - val_loss: 0.3655 - val_accuracy: 0.8667
Epoch 47/50
27/27 - 9s - loss: 0.0083 - accuracy: 1.0000 - val_loss: 0.3692 - val_accuracy: 0.8667
Epoch 48/50
27/27 - 9s - loss: 0.0081 - accuracy: 1.0000 - val_loss: 0.3612 - val_accuracy: 0.8667
Epoch 49/50
27/27 - 9s - loss: 0.0078 - accuracy: 1.0000 - val_loss: 0.3856 - val_accuracy: 0.8542
Epoch 50/50
27/27 - 9s - loss: 0.0074 - accuracy: 1.0000 - val_loss: 0.3784 - val_accuracy: 0.8625

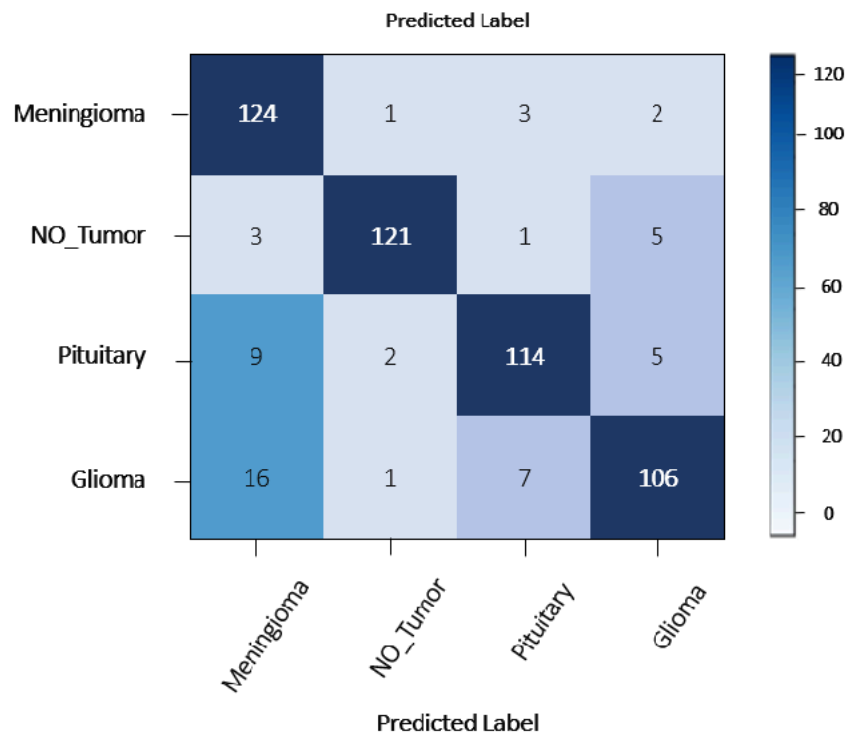
```

Figure 67: Training performance in accuracy and losses with transfer learning.

Source: Google Colab (Own creation)

Now, we observe that the system learns correctly, however, it produces a slightly lower performance than what we had when we implemented the neural network with 2 classes. The confusion matrix shows us that for some classes the predictions are not so exact.

1.



2.

```

[ ] evaluation = model.evaluate(train_batches)
    print('Test Accuracy: {}'.format(evaluation[1]))

59/59 [=====] - 35s 589ms/step - loss: 0.2294 - accuracy: 0.8817
Test Accuracy: 0.8817204236984253

```

Figure 68: a) Confusion matrix for 4-class CNN., b) Evaluation of accuracy in the training process.

Source: Google Colab (Own creation)

Figure 68a shows us the CNN predictions, as we see, we have lost precision in these predictions, the main justification for this effect is due to the physical similarity between classes, as we have mentioned, the main challenge of this work is having to manage the enormous similarity that exists as we can see in the following image. On the other hand, Figure 68b shows the precision in the training process which reaches 88.17%.

The accuracy of the predictions are as follows:

- **Meningioma:** 96.9%
- **No_Tumor:** 93,8%
- **Pituitary:** 90%
- **Glioma:** 83,8%

It is also important to mention that the system now loses precision as we add new classes, however, it is still a quite acceptable reading if we compare it with other works carried out with these same *datasets* (see section 4.3). Next, we will look at a real case that justifies what we said:

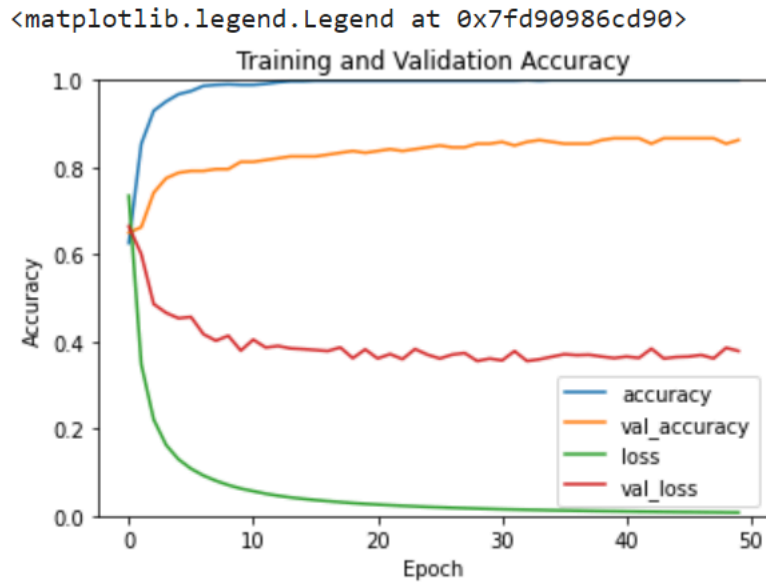


Figure 69: Performance graphs in accuracy and losses for the cnn training and validation process.

Source: Google Colab (Own creation).

On the other hand, Figure 69 reflects what was said above, the trend graphs of the training and validation of the system show that despite having a good learning process and losses during training, the graphs reflect the opposite during the validation process showing low performance, so we can assure that the results are not as good compared to the previous example since the predictions of some classes (mainly meningioma) are not very good.

MRI vista craneal posterior

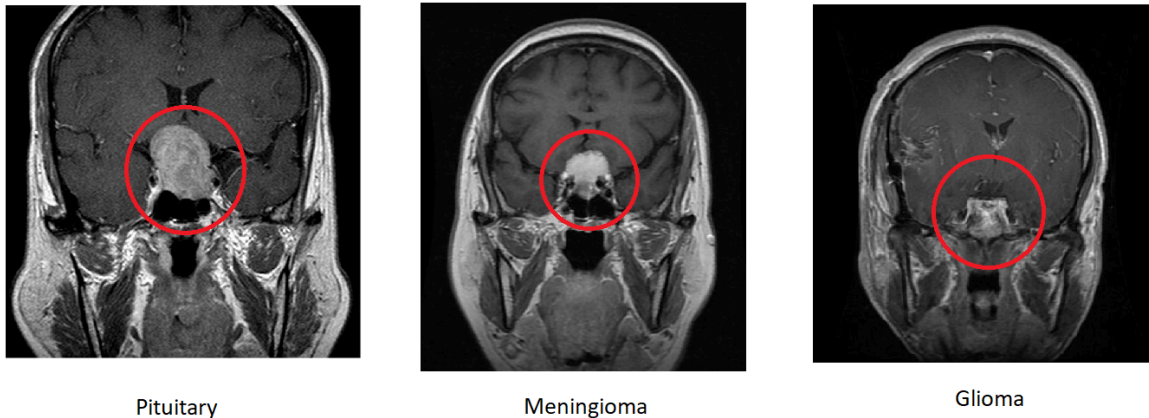


Figure 70: Similarity between images of different classes.

Source: Figshare Dataset (Available at: https://figshare.com/articles/dataset/brain_tumour_dataset/1512427)

Figure 70 shows the similarity that exists between the different types of tumours in some MRI images of the dataset, this is because the cancerous tissue is located in a similar position in the brain and the shape of the tumour is usually similar in all cases, this clearly impacts the precision of the neural network at the time of training, on the other hand, the resolution of some images in these datasets are not of the best quality, and although we have eliminated the most distorted images, there is still a percentage of images with deficiencies.

In addition to this, we must take into account that the datasets are a collection of images with different views as we see in Figure 71, which adds even greater heterogeneity.

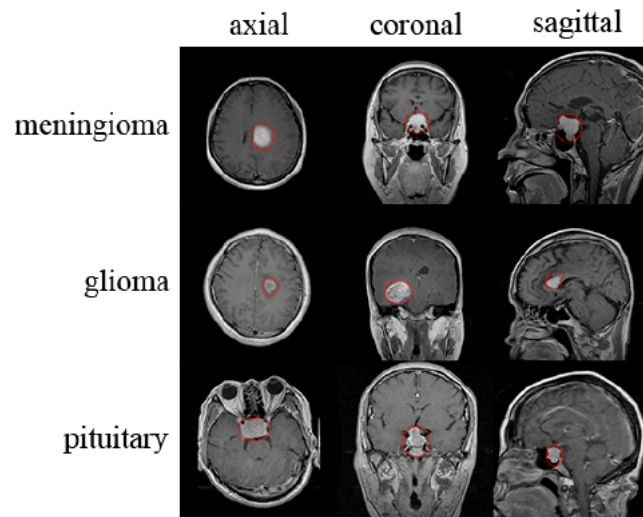


Figure 71: Views in MRI images from different angles in datasets.

Source: Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network (page 4)

This happens because tumours are not usually located in specific points in the brain tissue. On the contrary, different images must be taken from different views in order to have better criteria when making the final diagnosis. The medical analysis must take into account every detail in the anatomy of the tumour and that is why there is this variability in the images.

8.8 COMPARATIVE ANALYSIS WITH OTHER STUDIES

The results presented in the previous section show us that the performance is good if we take into account the details to which we refer and that represent the greatest challenge of this work, since the databases available in Libra form are very limited and there are few. The work we talked about in section 4.3 called "Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network" shows us that despite the efforts to adjust the neural network models as much as possible to achieve excellent results and with high precision, it is not a simple task and requires certain improvements in the datasets to achieve better results.

1.

Confusion matrix - testing data				
Outputs	meningioma	glioma	pituitary	
	873 74.0%	249 21.1%	57 4.8%	74.0% 26.0%
	334 10.9%	2620 85.7%	103 3.4%	85.7% 14.3%
	186 9.8%	35 1.8%	1684 88.4%	88.4% 11.6%
	62.7% 37.3%	90.2% 9.8%	91.3% 8.7%	84.3% 15.7%
	meningioma	glioma	pituitary	
Targets				

2.

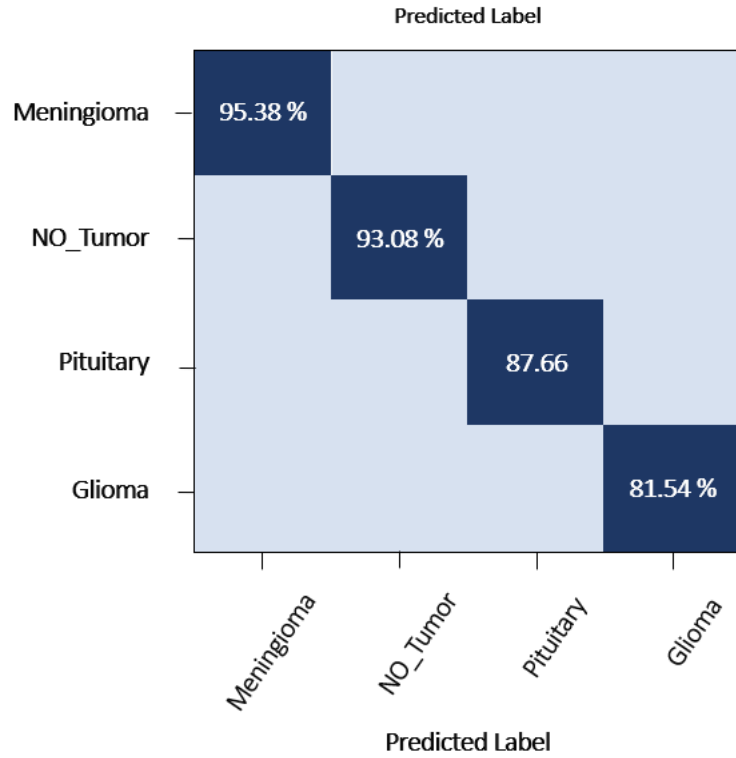


Figure 72: a) Confusion Matrix obtained in previous study. B) Percentage of true positive readings in our system predictions.

Source: Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network (page 8)

As we see in Figure 72a, the confusion matrix for the study “Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network” the precision obtained with data augmentation of the Figshare dataset shows a slightly lower performance than our case study, however, it is still very similar to our results shown in Figure 68, with the difference that the predictive model proposed in this study positions Meningioma as the class with the least precision at the output with 74.0% accuracy, followed by glioma with 85.7% and lastly the pituitary tumour with 88.4%. These results were obtained with the following neural network model:

Layer No.	Layer Name	Layer Properties
1	Image Input	256 × 256 × 1 images
2	Convolutional	16 5 × 5 × 1 convolutions with stride [2 2] and padding ‘same’
3	Rectified Linear Unit	Rectified Linear Unit
4	Dropout	50% dropout
5	Max Pooling	2 × 2 max pooling with stride [2 2] and padding [0 0 0 0]
6	Convolutional	32 3 × 3 × 16 convolutions with stride [2 2] and padding ‘same’
7	Rectified Linear Unit	Rectified Linear Unit
8	Dropout	50% dropout
9	Max Pooling	2 × 2 max pooling with stride [2 2] and padding [0 0 0 0]
10	Convolutional	64 3 × 3 × 32 convolutions with stride [1 1] and padding ‘same’
11	Rectified Linear Unit	Rectified Linear Unit
12	Dropout	50% dropout
13	Max Pooling	2 × 2 max pooling with stride [2 2] and padding [0 0 0 0]
14	Convolutional	128 3 × 3 × 64 convolutions with stride [1 1] and padding ‘same’
15	Rectified Linear Unit	Rectified Linear Unit
16	Dropout	50% dropout
17	Max Pooling	2 × 2 max pooling with stride [2 2] and padding [0 0 0 0]
18	Fully Connected	1024 hidden neurons in fully connected (FC) layer
19	Rectified Linear Unit	Rectified Linear Unit
20	Fully Connected	3 hidden neurons in fully connected layer
21	Softmax	softmax
22	Classification Output	3 output classes, “1” for meningioma, “2” for glioma, and “3” for a pituitary tumor

Figure 73: Neural Network Model applied in study.

Source: Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network (page 4)

This model is made up of a total of 22 layers, while our original design was made up of a total of 17 as shown in Figure 38, *where we* clearly notice that despite the increase in data and the design of the network, the learning reaches a saturation point that, in the case of the *datasets* that we have used, even when we try to continue applying the data augmentation *technique* and further adjusting the CNN, we will not achieve greater changes, for which we can recommend the use of *more* diverse datasets and with higher image resolution quality with the aim of having greater precision in the output.

While Figure 72b *shows us* the percentages of true positive readings that our system has, as we see the reading with the lowest percentage of accuracy is still slightly better than the one proposed in the study by Milica M. Badža and Marko Č. Barjaktarovic, which makes it clear that the increase in data can become saturated without producing improvement effects.

CHAPTER IX: APPLICATION DEVELOPMENT

Once we have managed to build a system whose output shows reproducible results and of course, with the expected performance, we will proceed to build a web application that will give added value based on the recommendations obtained from the Association of People Affected by Brain Tumours in Spain (ASATE), in which specialists assure that since it is a very sensitive medical field and in which a lot of experience is required in both didactic and practical knowledge, we are not yet at the appropriate level to trust 100% in the predictions made by a neural network, even when it is highly accurate.

An error in prediction, no matter how small, could have negative consequences for the patient, firstly because the treatment for one or another tumour is not the same in all cases, secondly because the physical location in the brain could be very similar between the different types of tumours and this could yield misleading predictions and thirdly because a multidisciplinary study is required to give an exact diagnosis, where specialists from the area of Radiology, Oncology and Neuroscience must be involved.

9.1 PRE-PROFESSIONAL FEEDBACK

To obtain an opinion that supports our study, we have interviewed the Medical advisory staff of the Association of People Affected by Brain Tumours of Spain (ASATE), who have given us their opinion from a medical and operational point of view, with the aim of building a system that is beneficial for those who would use this system, the main points that we have obtained from the interview we have already studied previously and other secondary opinions are presented below:

- Brain tumours have been studied for a long time, this study has been evolving as new technological trends appear in different areas such as computer vision, assisted surgical procedures, assistive robotics for cancer detection and treatment.
- The medical community must rely on a series of recommendations established by the World Health Organization (WHO) and government entities when it comes to procedures where intervention by an anesthesiologist, surgeon or any specialist in invasive interventions and/or provision of medications to patients is required.
- Our proposal is about a new system not yet tested in health centers at a local or international level.
- The use of neural networks for brain tumour prediction supports research, however, it cannot yet be used as an official procedure for medical diagnosis.
- The system that we have proposed could be used as a suggestion to support the medical personnel in charge of giving diagnoses, this would shorten times in the analysis of images to give a diagnosis.
- The predictions given by the system could not only help reduce waiting times, but could also reduce human error in interpretation in the first phase of the analysis.
- The most standardized applications use web services, due to their ease of use and the wide range of functions that can be implemented. Likewise, with the rise of cloud services offered for storage, exclusive and dedicated databases can be used to store the images obtained and thus be able to enable other attached services.
- Because in most medical centers, MRIs are usually stored in local databases, the first analyzes are carried out on digital images, applying different filters to be able to analyze different scenarios according to shape, size, etc.

9.2 APPLICATION DEVELOPMENT

Thanks to this technical input, it has been decided to develop a simple application capable of providing the predictions made by our model. For this we will use a tool called *Flask*, which is a Python microframework, as explained in section 5.10 in which our web service will make an HTTP request to a web server that will store our created model and which, in turn, will give us the result of the predictions about the image that we want to analyze.

This is basically a request and response service, in which our back-end will be the Flask service and the front-end will be the web application through the HTTP and JAVASCRIPT languages, so what the user will see will be the predictions of the model from the web, but the programming that is hidden from the user will be the Python microframework that we do not see from the user's side. Figure 71 shows the operation of the proposed application.

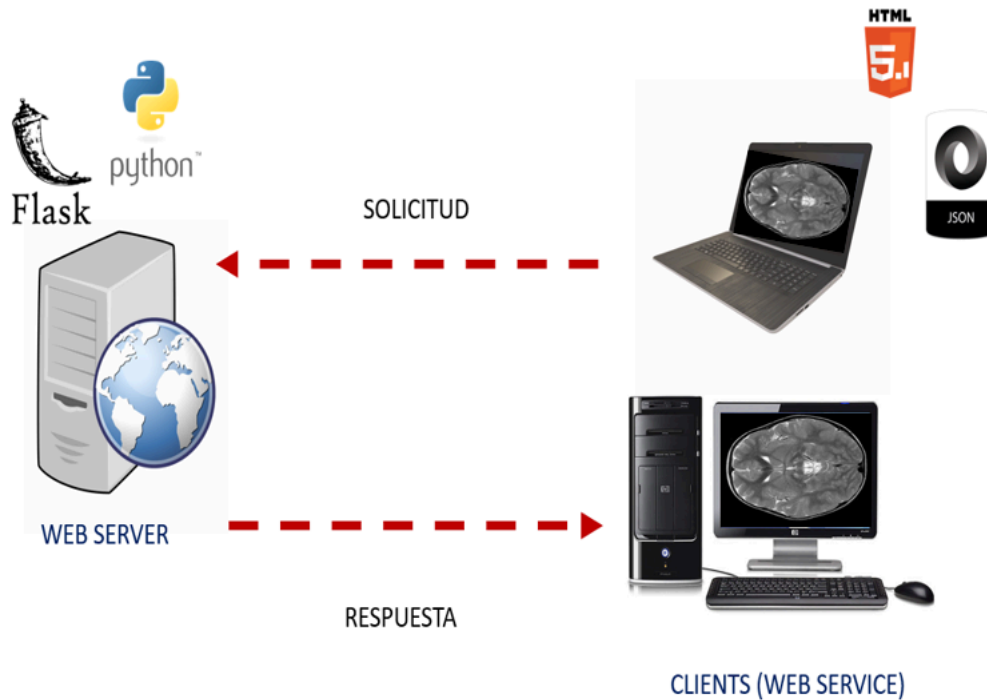


Figure 74: Operating diagram of the proposed application.

Source: Own elaboration.

To enable the use of this web application, we must take into account that the user does not have knowledge of the programming language in which the neural network model was developed and in fact, he should not know it since his interest is focused on the web service through which he will obtain the predictions of the neural model, this means that we must create a system capable of interpreting our model (independent of the language) which we have obtained in the Google Colab *programming* environment, the web application cannot be interpreted in the Colab environment.

9.2.1 BACK-END APPLICATION DEVELOPMENT.

Previously we discussed the structure of our application (See Figure 74), which is based on two separate applications, in this case we will talk about the back end, which is nothing more than our Flask application that is written in Python, we have called this application `app_cnn_predict4.py` which calls our model previously obtained from our neural network whose name is `Model_4clases.h5`. To do this, we have created an environment within our Jupyter Notebooks repository with the Flask apps as we see in the following Figure 75.

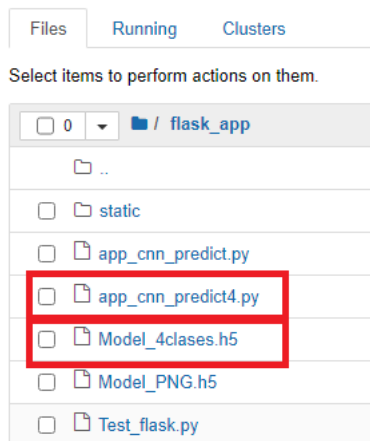


Figure 75: Flask application and neural network model repository.

Source: Own elaboration (Jupyter Notebook development environment)

Once this model loads, we pre-process the image to define size, color scale and convert the image to an array, then we do an encoding that is assigned to an image with the key called 'image' from JSON which is stored in the message variable, to later decode said image in `io.BytesIO` within the encoded image, not in memory, then we define the possibilities per class to make the prediction through a NumPy array, the aforementioned is shown in the following Figure 76.

```
def get_model():
    global model
    model = load_model('Model_4clases.h5')
    print(" * Modelo cargado correctamente!")

def preprocess_image(image, target_size):
    if image.mode != "RGB":
        image = image.convert("RGB")
    image = image.resize(target_size)
    image = img_to_array(image)
    image = np.expand_dims(image, axis=0)
    return image
```

Figure 76: Image pre-processing within the Flask application called `app_cnn_predict4.py`

Source: Own elaboration (Jupyter Notebook development environment)

9.2.2 FRONT-END APPLICATION DEVELOPMENT

The front end will be the complement of our web application written in HTML, whose function is to call the Flask application that contains the neural network model that we have built, to enable the Flask environment, we do it as shown in the following Figure 77.

```

C:\Users\personal\flask_app>set FLASK_APP=app_cnn_predict4.py
C:\Users\personal\flask_app>python -m flask run --host=0.0.0.0
 * Serving Flask app "app_cnn_predict4.py" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off

 * Modelo cargado correctamente!
 * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
 * Running on http://10.211.55.11:5000/ (Press CTRL+C to quit)
127.0.0.1 - - [18/Sep/2021 14:30:45] "GET /static/cnn_predict4.html HTTP/1.1" 304 -
127.0.0.1 - - [18/Sep/2021 14:30:46] "GET /favicon.ico HTTP/1.1" 404 -

```

Figure 77: Enabling Flask environment.

Source: Own creation (CMD Windows 10 console)

Once our Flask environment is enabled, within our HTML application we will use a small and simple JavaScript library, but with a large number of functions called JQuery (jQuery, 2006), the repository is available at: <https://jquery.com/download/> in which we will use the API available at: <https://code.jquery.com/jquery-3.3.1.min.js>, which will allow us to call a function called prediction that contains the image of each class, as we see in Figure 78.

```

<p style="font-weight:bold">Selecciona la imagen en formato .png y pulsa boton de prueba</p>
<input id="image-selector" type="file">
<button id="predict-button">Prueba</button>
<p style="font-weight:bold">Prediccion 4 clases</p>
<p> Meningioma: <span id="men-prediction"></span></p>
<p> NO Tumor: <span id="notu-prediction"></span></p>
<p> Glioma: <span id="gli-prediction"></span></p>
<p> Pituitary: <span id="pit-prediction"></span></p>
<img id="selected-image" src="" />

<script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
</script>

```

Figure 78: Access to the JQuery library to call the "prediction" function

Source: Own elaboration (Jupyter Notebook development environment)

Finally, we will tell our app to make a "POST" to publish the predictions of each of the classes and thus see numerically each of the probabilities that an image has of presenting one or another type of Tumor, as shown in Figure 79.

```

console.log(message);
$.post("http://127.0.0.1:5000/cnn_predict4", JSON.stringify(message), function(response){
    $("#men-prediction").text(response.prediction.men.toFixed(6));
    $("#notu-prediction").text(response.prediction.notu.toFixed(6));
    $("#gli-prediction").text(response.prediction.gli.toFixed(6));
    $("#pit-prediction").text(response.prediction.pit.toFixed(6));
    console.log(response);
});

```

Figure 79: Publication in a web environment in the app written in HTML.

Source: Own elaboration (Jupyter Notebook development environment).

We will store these repositories within a folder that we have called "static" which contains our HTML app, as we see in Figure 80.

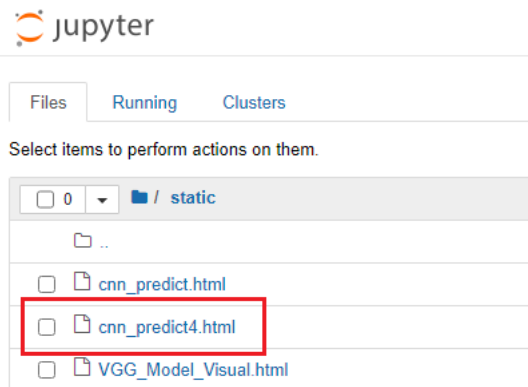


Figure 80: HTML application repository within our master repository.

Source: Own elaboration (Jupyter Notebook development environment).

9.2.3 EXECUTION OF THE WEB APPLICATION.

In the previous sections the development of our application was explained, now we will demonstrate the results once we have enabled the app, to do this we access the Flask web server through the address: `http://127.0.0.1:5000/static/cnn_predict4.html`, and the following interface that we see in Figure 81 will open.

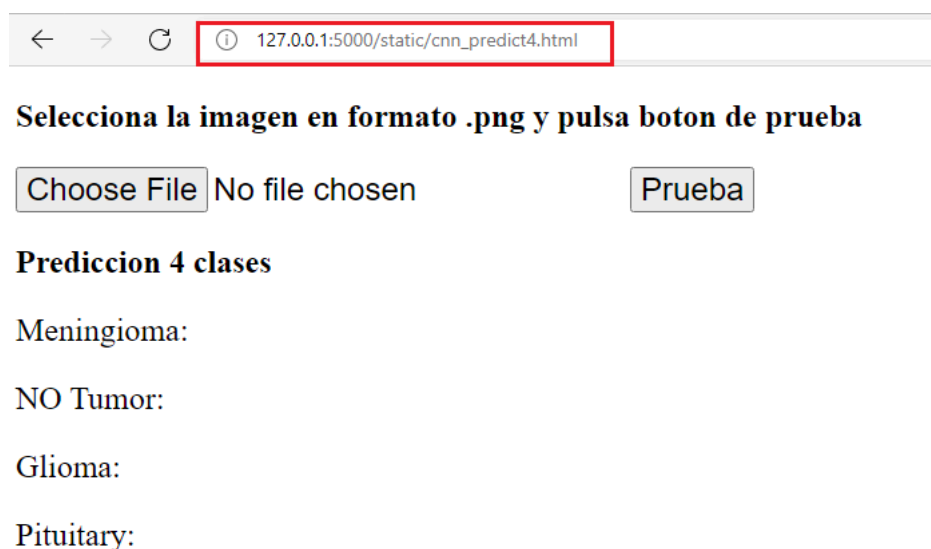


Figure 81: Access to the web application.

Source: Own elaboration (http environment).

When choosing an image that can be stored within our computer or server with the MRI images *obtained* from some *dataset*, we will proceed to press the Test button and we will obtain the results of the predictions by class. We can see the above in Figure 82, which represents the predictions for each of the classes in different MRI images, as follows: a) Meningioma, b) Non-Tumor, c) Glioma and d) Pituitary.

1.

← → ↻ ⓘ 127.0.0.1:5000/static/cnn_predict4.html

Selecciona la imagen en formato .png y pulsa boton de prueba

Choose File meninglioma (2).png Prueba

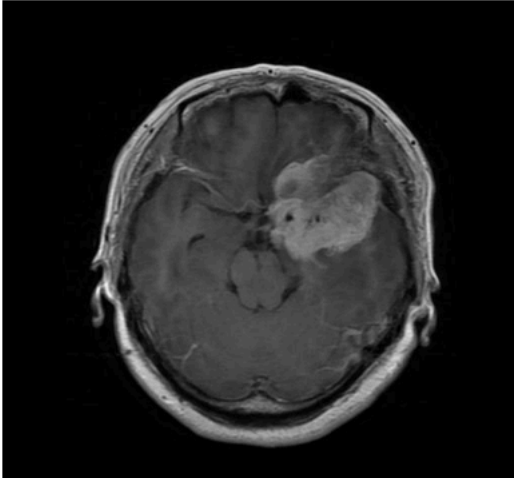
Prediccion 4 clases

Meningioma: 0.999955

NO Tumor: 0.000030

Glioma: 0.000014

Pituitary: 0.000002



2.

← → ↻ ⓘ 127.0.0.1:5000/static/cnn_predict4.html

Selecciona la imagen en formato .png y pulsa boton de prueba

Choose File NOT (62).png Prueba

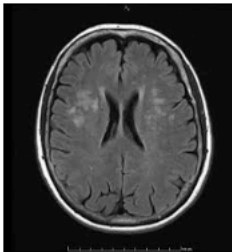
Prediccion 4 clases

Meningioma: 0.000086

NO Tumor: 0.999819

Glioma: 0.000094

Pituitary: 0.000001



3.

← → ↻ ⓘ 127.0.0.1:5000/static/cnn_predict4.html

Selecciona la imagen en formato .png y pulsa boton de prueba

glioma (23).png

Prediccion 4 clases

Meningioma: 0.038257

NO Tumor: 0.018373

Glioma: 0.943328

Pituitary: 0.000042



4.

← → ↻ ⓘ 127.0.0.1:5000/static/cnn_predict4.html

Selecciona la imagen en formato .png y pulsa boton de prueba

pituitary (654).png

Prediccion 4 clases

Meningioma: 0.007176

NO Tumor: 0.000283

Glioma: 0.051178

Pituitary: 0.941363

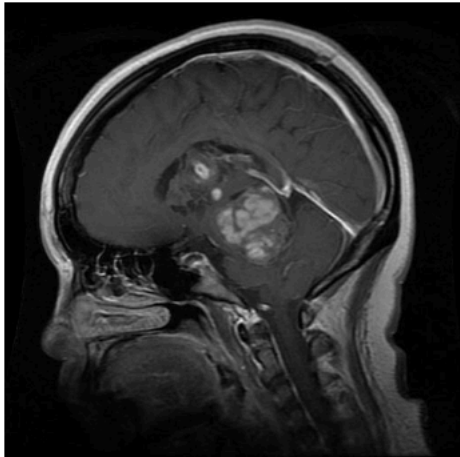


Figure 82: Predictions obtained in our web application.

Source: Own elaboration (http environment).

Figure 82 shows us an example of prediction for each class. In it we have been able to verify that the accuracy in the training and validation process demonstrates consistent results for each class. To carry out a test we have taken images in which we previously knew their class and we have challenged the system to check the predictions and it has a high reading precision.

9.2.4 VISUAL APPLICATION EXECUTION

In addition to our web application, we have developed another application that allows you to see the results of the predictions with a visual format in which bar and pie charts are shown to have a slightly more intuitive idea of each of the predictions.

To use the D3.js Java Script libraries available for free on the web, this tool allows us to add value to our Machine Learning projects. We know that the visualization of results is not only useful for interpreting the effects of the system on the output, but it is also a more dynamic way to see these results. D3.js allows you to manipulate documents based on data and is friendly to HTML, SVG and CSS languages, thus combining visualization and data components (D3.js, 2011).

```
<script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
<script src="https://d3js.org/d3.v5.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/crossfilter/1.3.12/crossfilter.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/dc/3.0.3/dc.min.js"></script>
<script>
```

Figure 83: Enabling library for D3.js functionality for graphical application.

Source: Own elaboration (http environment).

Figure 83 shows how we have enabled the D3.js and CDJn libraries for the application we have proposed. Next, we will see the results.

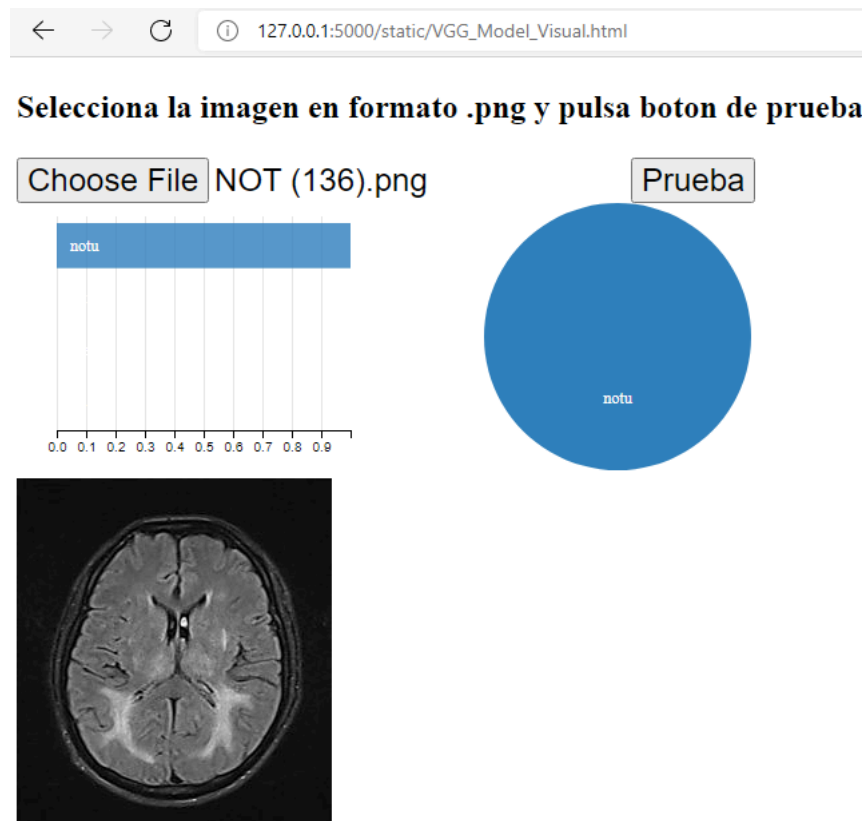


Figure 84: Visual application for class prediction. Example with an MRI image without tumor.

Source: Own elaboration (http environment).

Figure 84 allows us to appreciate the result that we obtain at the output of our visual web application, in it we have selected an image that represents a brain without tumour, for which the result reflected in both the horizontal bar graph and the pie graph, show that the image has a 99.99% probability that it is a brain without tumour, this corresponds to the results obtained in the application of predictions that we see in Figure 85.

Selecciona la imagen en formato .png y pulsa boton de prueba

NOT (136).png

Prediccion 4 clases

Meningioma: 0.000001

NO Tumor: 0.999999

Glioma: 0.000000

Pituitary: 0.000001

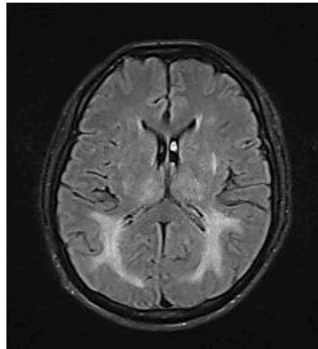


Figure 85: Numerical results of predictions on the same image used in the previous example.

Source: Own elaboration (http environment).

Now, we will analyze an image with a slightly higher confusion index, in which an MRI image of a pituitary tumour has a lot of similarity to other classes, due to the position where the cancerous tissue is located, the size of the tumour, and the shape. We analyze the result below.

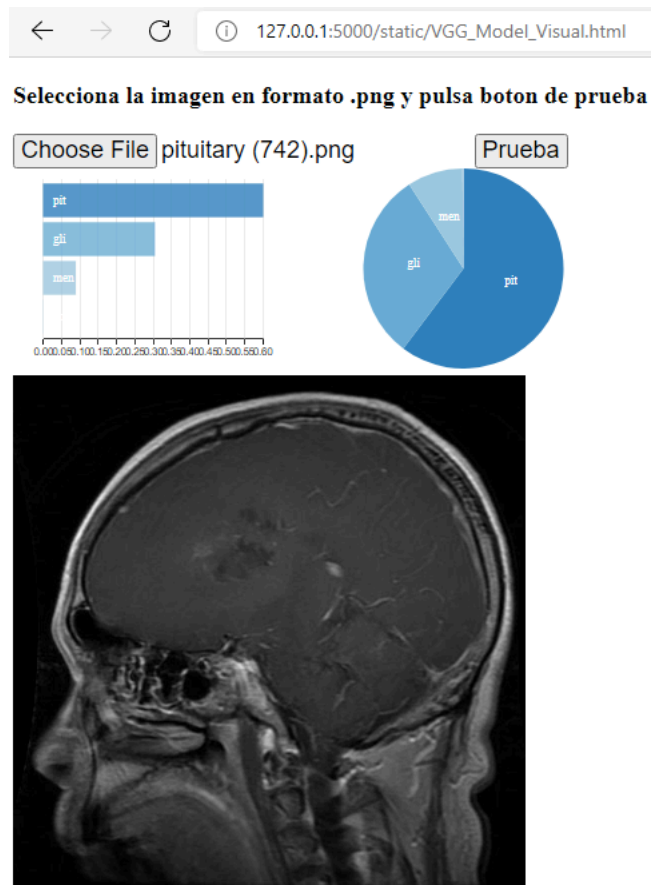


Figure 86: Visual application for a Pituitary Tumour with similarity to other classes.

Source: Own elaboration (http environment).

We observe in Figure 86 that although the MRI image of a Pituitary tumour has high similarity to another, the predictions are still compatible with the predictions in an accurate way.

Selecciona la imagen en formato .png y pulsa boton de prueba

pituitary (742).png

Prediccion 4 clases

Meningioma: 0.089641

NO Tumor: 0.001447

Glioma: 0.306253

Pituitary: 0.602659

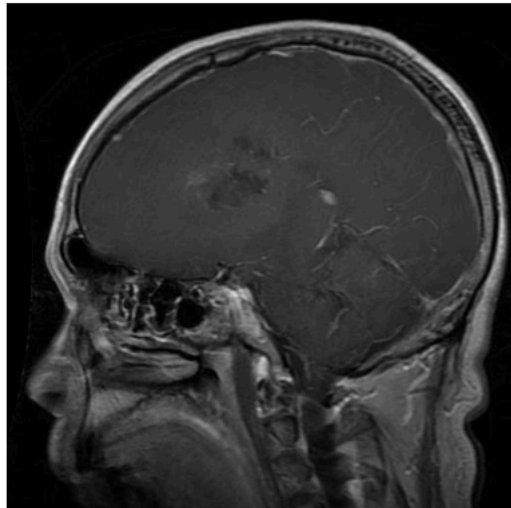


Figure 87: Numerical analysis of predictions for the same MRI image with Pituitary Tumour.

Source: Own elaboration ([http environment](http://environment)).

As we see in Figure 87, we see that the predictions show a 60.26% probability for the pituitary tumour and the remaining percentage of 39.74% classes are made up of the other classes, which demonstrates the compatibility of the results since the highest probability is for the Pituitary tumour.

CONCLUSIONS.

The development of this work provides knowledge in different technological and scientific areas, with the main intention of helping society in general in a vital area such as health. The idea is that in the future the diagnostic processes will be faster and more efficient, allowing patients who suffer from this type of illness to obtain their medical therapy as soon as possible in order to avoid complications in the future, since the fundamental right of each individual within society to improve their quality of life must be feasible for everyone without exception.

And on the technical side we have also seen that although there is not a vast amount of databases available for research purposes, very positive results can be obtained; however, the opportunity for improvement is very great, not only at the level of the datasets, but also at the level of Deep Learning techniques that allow obtaining even more reliable results.

With the completion of this project, we can ensure that the objectives proposed at the beginning of the work have been met, since not only have good results been achieved with the material available to train our neural network, but we have added additional value with the implementation of a web application applicable in a real scenario and that will ultimately help medical personnel a lot with the medical diagnosis process.

Additionally, some of the main technical observations that we can capture with this work are:

- The datasets available for free provide enough information to train a neural network with acceptable results. These datasets can be combined to produce a better effect on the results, since it allows the amount of information available for the training and validation processes of our neural network.
- The Transfer Learning technique shows that it is extremely useful for training processes of a neural network; the results reflect an important level of accuracy and, therefore, reliability.
- The Data Augmentation technique is also of utmost importance when what is sought is better prediction results.
- The functions offered by Keras and TensorFlow are very useful when working with neural networks for Machine and Deep Learning.
- The use of Google Colab has been fundamental for the development of this work, since thanks to this tool we have been able to make a very powerful GPU, which has greatly facilitated the optimization of training time, without leaving aside the fact that it is free and we can integrate it with Google Drive, so this gives us the advantage of portability, where we do not depend on a single workstation, but we can use any computer with an internet connection, without having to download the datasets on each workstation.
- The combination of datasets has allowed us to improve the accuracy of the results.
- The creation of a neural network model has allowed us to bring the predictions made by the system to a web application, which is closer to the reality of a work scenario in a medical center.
- The implementation of Flask has been very useful to us since we have used Python as a development language, and Flask, being written in Python, allows us to create an application in the same environment.
- The implementation of our neural network has shown us that although brain tumours have great heterogeneity, reliable prediction models can be built, which makes it clear to us that regardless of the types of images we are using, a fine adjustment can always be made in order to improve the quality of the predictions.
- As in any technological and scientific area, the possibility of improving results can always be evaluated. In our case, it is clear that improving the quality of the datasets would be a huge opportunity to create even more reliable systems.

NEXT STEPS TO TAKE

As we have stated in the conclusions, it is also necessary to say that a possible second stage of this project to be implemented in the future includes some improvements that we present below:

- It is recommended to improve the quality of the datasets; this task requires collaboration on the part of medical staff. Proper management of the datasets would positively impact the results and, therefore, the precision and reliability would be better.
- While it is true, the task of medical diagnosis of brain tumours requires a deep multidisciplinary analysis, however, if prediction results can be improved, we would be on the verge of changing some techniques for the medical diagnosis of this type of cancer.
- Likewise, the implementation of this proposal can be put into practice in the prediction of other types of tumours where the anatomy could vary; however, with the appropriate datasets, this model could be used to build a robust neural network with excellent results.
- With the rise of mobile applications, models with mobile functionality could also be created.
- The use of other medical image formats could also provide a considerable improvement to the training processes of a neural network. As is known, other techniques have recently been implemented for obtaining medical images that could be implemented in the area of neuroscience, an extremely complex area due to the anatomical nature of the central nervous system and its great importance in the performance of human physiological functions.

REFERENCES

- Ali Habib, M. (August 31, 2019). Github. Retrieved from Brain Tumor Detector with Tensorflow & Keras: <https://github.com/MohamedAliHabib/Brain-Tumor-Detection>
- Argibay, P. (2011). *Advanced statistics: The problem of overfitting and the method of minimal descriptions*. Buenos Aires, Argentina: Magazines of the Italian Hospital.
- ASATE. (February 21, 2021). Association of People Affected by Brain Tumours in Spain. Obtained from Did You Know: <http://www.asate.es/tumores-cerebrales/sabias-que/>
- Braininvestigators. (October 8, 2018). *Mind Decoding Co*. Retrieved from <https://www.braininvestigators.com/neurociencia/inteligencia-artificial-tandem/>
- Chakraborty, N., & Kanchan, S. (May 24, 2020). Kaggle. Retrieved from Brain Tumour Classification (MRI): <https://www.kaggle.com/sartajbhuvaji/brain-tumour-classification-mri>
- Cheng, J. (April 2, 2017). Brain Tumour Dataset Figshare. Retrieved from <https://doi.org/10.6084/m9.figshare.1512427.v5>
- Cheng, J., Huang, W., Cao, S., Yang, R., Yang, W., Yun, Z., . . . Feng, Q. (2015). Enhanced Performance of Brain Tumour Classification via Tumor Region Augmentation and Partition. Southern Medical University. Guangzhou, China: Southern Medical University.
- D3js. (February 11, 2011). *Data-Driven Documents*. Retrieved from <https://d3js.org/>
- Flores López, R., & Fernández F, J. (2008). *Artificial Neural Networks: Theoretical Foundations and practical applications*. León: Netbiblio.
- García Benítez, S., López Molina, J., Ramos Trejo, O., Palacios Morales, I., & Mora Méndez, L. (2016). *Neural Networks in geotechnical exploration and parametric modeling: towards more competitive deliverables*. Mérida, MX: Mexican Society of Geotechnical Engineering.
- García Fenoll, I. (2010). *Contributions to the segmentation and characterization of 3D medical images*. Seville: University of Seville.
- Gili, J. (2002). Biophysical Introduction to Magnetic Resonance in Neuroimaging. In J. Gili. Barcelona: Scribd.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. Massachusetts: The MIT Press.
- Google Inc. (February 2018). *Google Collab*. Retrieved from Jupyter Notebook Projects: https://colab.research.google.com/notebooks/intro.ipynb?utm_source=scs-index
- Grinberg, M. (2014). Flask Web Development. In *Developing Web Application with Python* (pp. 3-4). Sebastopol, CA: O'Reilly Media Inc.
- IAARbook. (March 2018, 2018). IAARBook Github. Retrieved from <https://iaarbook.github.io/deeplearning/>
- Ibrahim, K., & Castelli, M. (2020). *The effect of batch size on the generalizability of the convolutional neural networks on a histopathology dataset*. Lisboa, Portugal: ScienceDirect.
- jQuery. (March 2, 2006). *jQuery Documentation*. Retrieved from <https://jquery.com/download/>
- Kaggle. (2010). *Kaggle Datasets*. Retrieved from www.kaggle.com/
- Kang, Y., Cho, N., Yoon, J., Park, S., & Kim, J. (2021). *Transfer Learning of a Deep Learning Model for Exploring Tourists' Urban Image Using Geotagged Photos*. Seoul, Korea: Department of Social Studies, Ewha Womans University, Seoul.
- Keepcoding. (2018). *Feature Engineering Made Easy*. Mumbai: Packt. Retrieved from <https://keepcoding.io/blog/que-son-datasets/>
- Keras. (March 2015). *Keras About*. Retrieved from <https://keras.io/about/>
- Keras Doc. (March 27, 2015). *Image data preprocessing*. Retrieved from Keras: <https://keras.io/api/preprocessing/image/>
- Milica M. , B., & Barjatarovic, M. (2020). Classification of Brain Tumours from MRI Images Using a Convolutional Neural Network. Belgrade, Serbia: University of Belgrade.
- NVIDIA. (March 2019). *NVIDIAUK*. Retrieved from <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf>
- Oracle Inc. (2016). *Oracle*. Obtained from <https://www.oracle.com/es/big-data/what-is-big-data/>
- World Health Organization. (2016). Department of Surgery, Radiology and Physical Medicine. Obtained from <http://www.oc.lm.ehu.es/Departamento/OfertaDocente/Teledocencia/Cruces/Cirugia1/NcE0Z.%20Clasificaci%C3%B3n%20tumores%20SNC%20OMS%202016%20Imprimer.pdf>
- Ozdemir, S., & Susarla, D. (2018). *Feature Engineering Made Easy*. Mumbai: Packt.
- Reyes Oliveiros, F., & Lema Bouzas, M. (2007). *Gliomas of the brain*. Santiago de Compostela: Pedro Barrié de la Maza Foundation.
- Siegel, R., Miller, K., & Jemal, A. (January 5, 2017). *American Cancer Society*. Retrieved from <https://acsjournals.onlinelibrary.wiley.com/doi/full/10.3322/caac.21387>
- Spanish Society of Medical Oncology. (December 18, 2019). SEOM Brain Tumours. Obtained from <https://seom.org/info-sobre-el-cancer/tumores-cerebrales?start=2>
- Tammina, S. (2019). *Transfer learning using VGG-16 with Deep Convolutional Neural Network for Classifying Images*. Hyderabad, India: International Journal of Scientific and Research Publications (IJSRP).
- TensorFlow. (2015). *Large-scale machine learning on heterogeneous systems*. Obtained from <https://www.tensorflow.org/?hl=es-419>
- Trincado Martínez, P. (October 2003). *Las Condes Clinic*. Obtained from Endocrinology Unit: http://www.clc.cl/clcprod/media/contenidos/pdf/MED_14_1/TumoresPituitarios.pdf
- USA National Cancer Institute. (March 26, 2020). Cancer Research Center. Retrieved from NCI-Connect: <https://www.cancer.gov/rare-brain-spine-tumour/tumours/meningioma>
- Wiemels, J., Wrensch, M., & Claus, E. (2010). *Epidemiology and etiology of meningioma*. San Francisco, CA: Springer.
- Wong, S., Gatt, A., Stamatescu, V., & McDonnell, M. (2016). *Understanding data augmentation for classification: when to warp?* Edinburgh, Australia: Information Technology and Mathematical Sciences University of South Australia.
- Wu, J. (2017). *Introduction to Convolutional Neural Networks*. Nanjing, China: Nanjing University, China LAMDA Group.

APPENDIX A. GLOSSARY

ASATE: Association of People Affected by Brain Tumours in Spain.

SEOM: Spanish Society of Medical Oncology.

WHO: World Health Organization.

MRI: Magnetic Resonance Imaging.

CT: Computerized Axial Tomography.

SPECT: Single-Photon Emission Computed Tomography.

PET: Positron Emission Tomography

IA: Inteligencia Artificial.